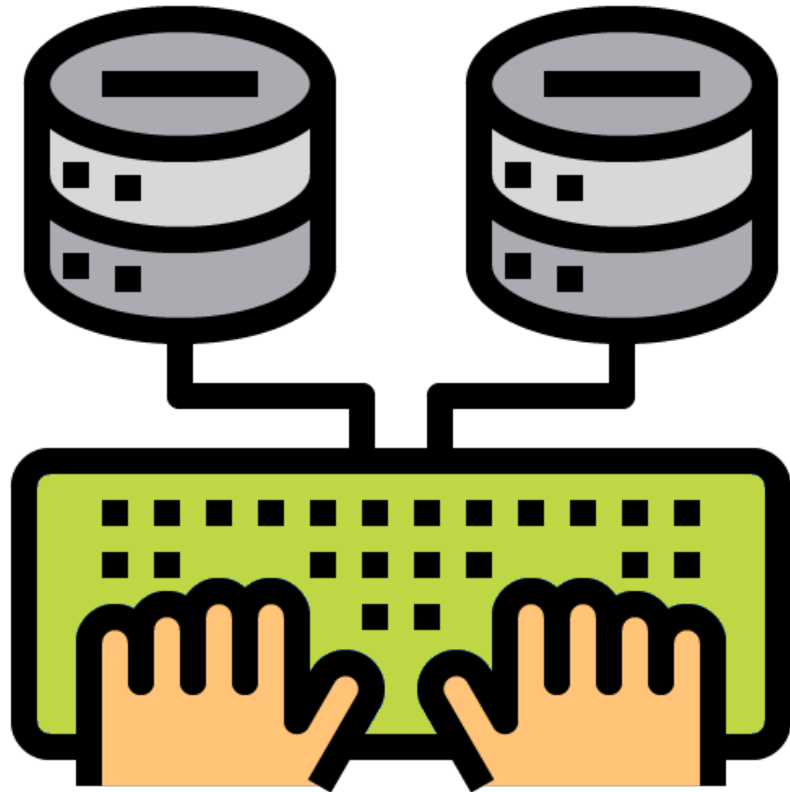




Big Data Analysis

Distributed Processing Big Data Analysis & MLLib

Lecturer:

Márcia Lourenço Baptista
m.baptista@novaims.unl.pt

Big Data Analysis

Big Data Analysis is about **analysing** the big data to extract patterns, and trends.

Key Activities:

- Statistical analysis
- Machine learning
- Data mining
- Predictive modeling
- Visualization & reporting



Big Data Analysis

Big Data Analysis is about **analysing** the big data to extract patterns, and trends.

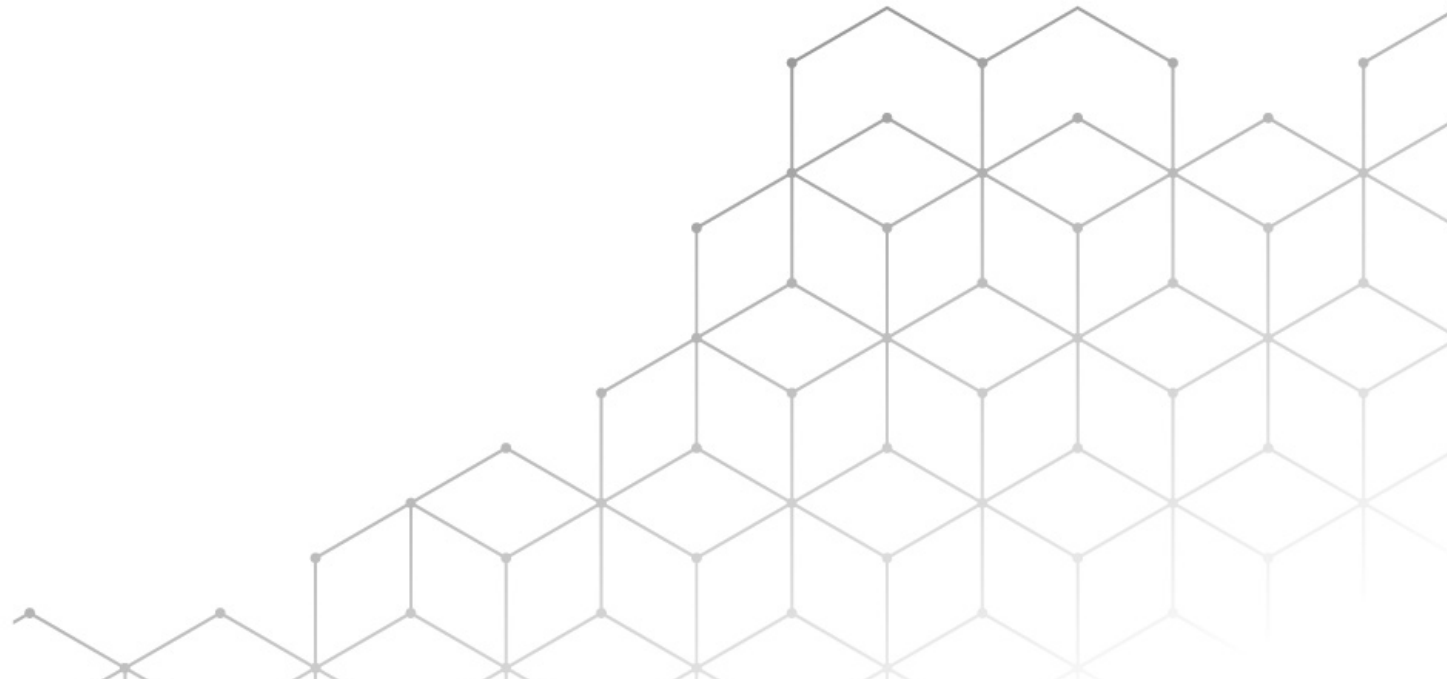
Key Activities:

- Statistical analysis
- Machine learning
- Data mining
- Predictive modeling
- Visualization & reporting



Learning Objectives

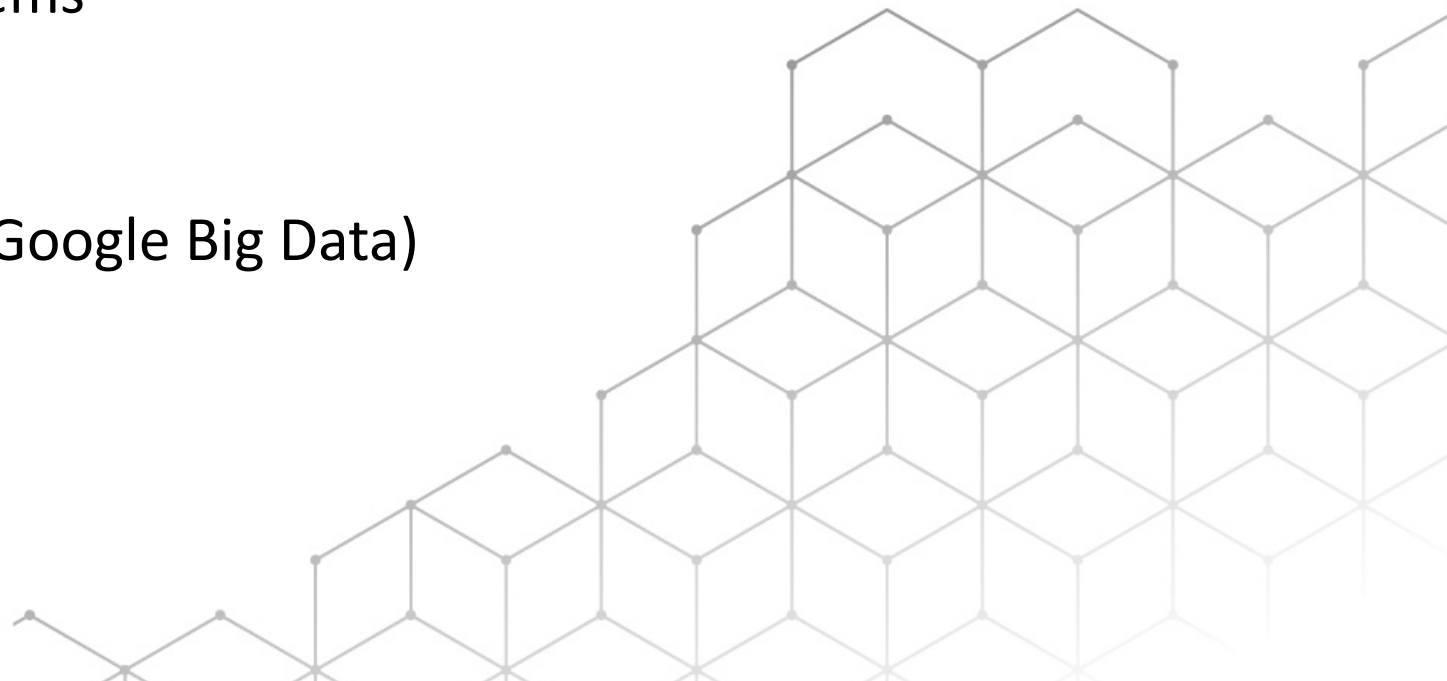
- Today we have the following agenda:
 - Big Data Analysis
 - Spark MLlib



Agenda for Today



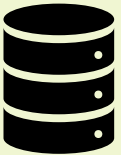
- Big Data Analysis: how to do machine learning on top of big data?
 - Remember what is big data?
 - What is big data analysis?
 - Terminology of big data analysis
 - Three types of learning problems
 - Process of big data analysis
 - Tools of big data analysis
 - Example of big data analysis (Google Big Data)
- Machine Learning
- Spark MLlib



What is big data?

- Big Data refers to a huge volume of data, that cannot be stored and processed using the traditional computing approach within a given time frame.

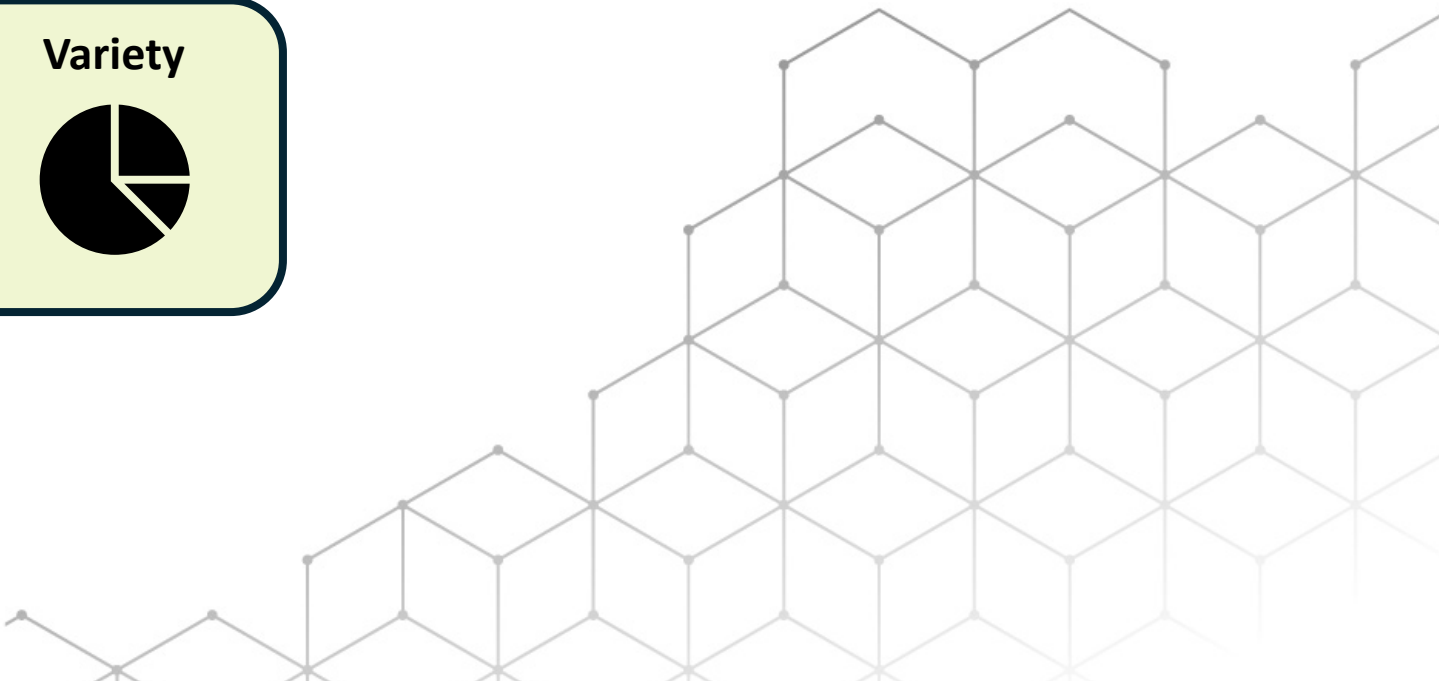
Volume



Velocity

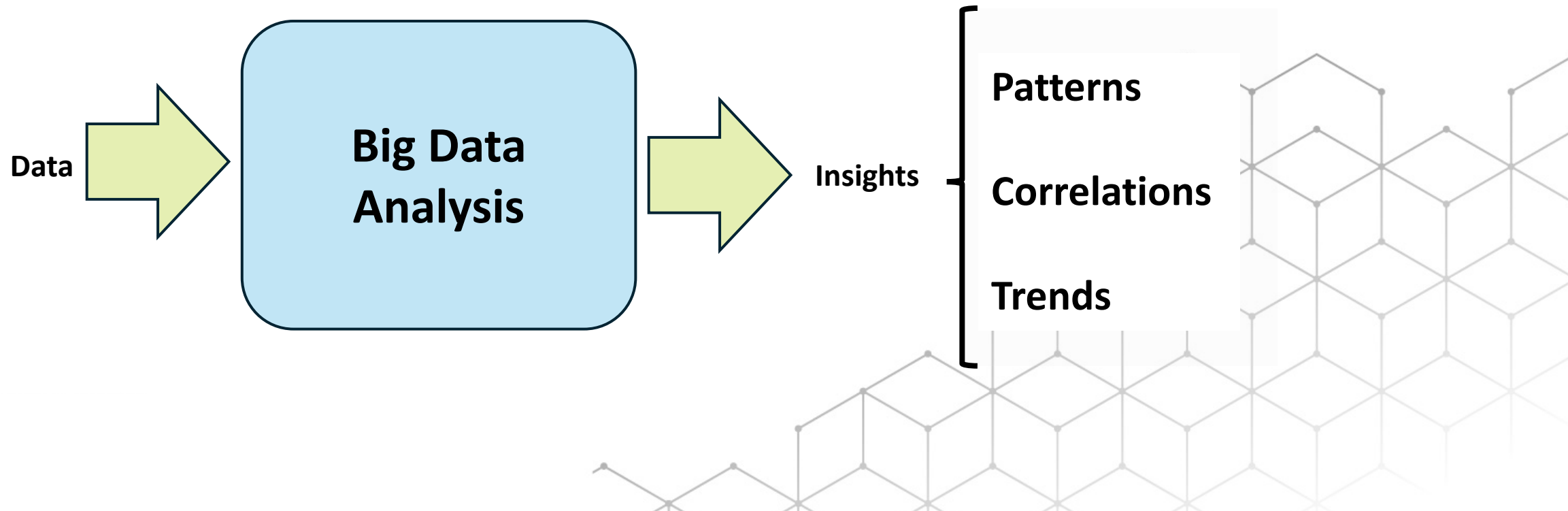


Variety



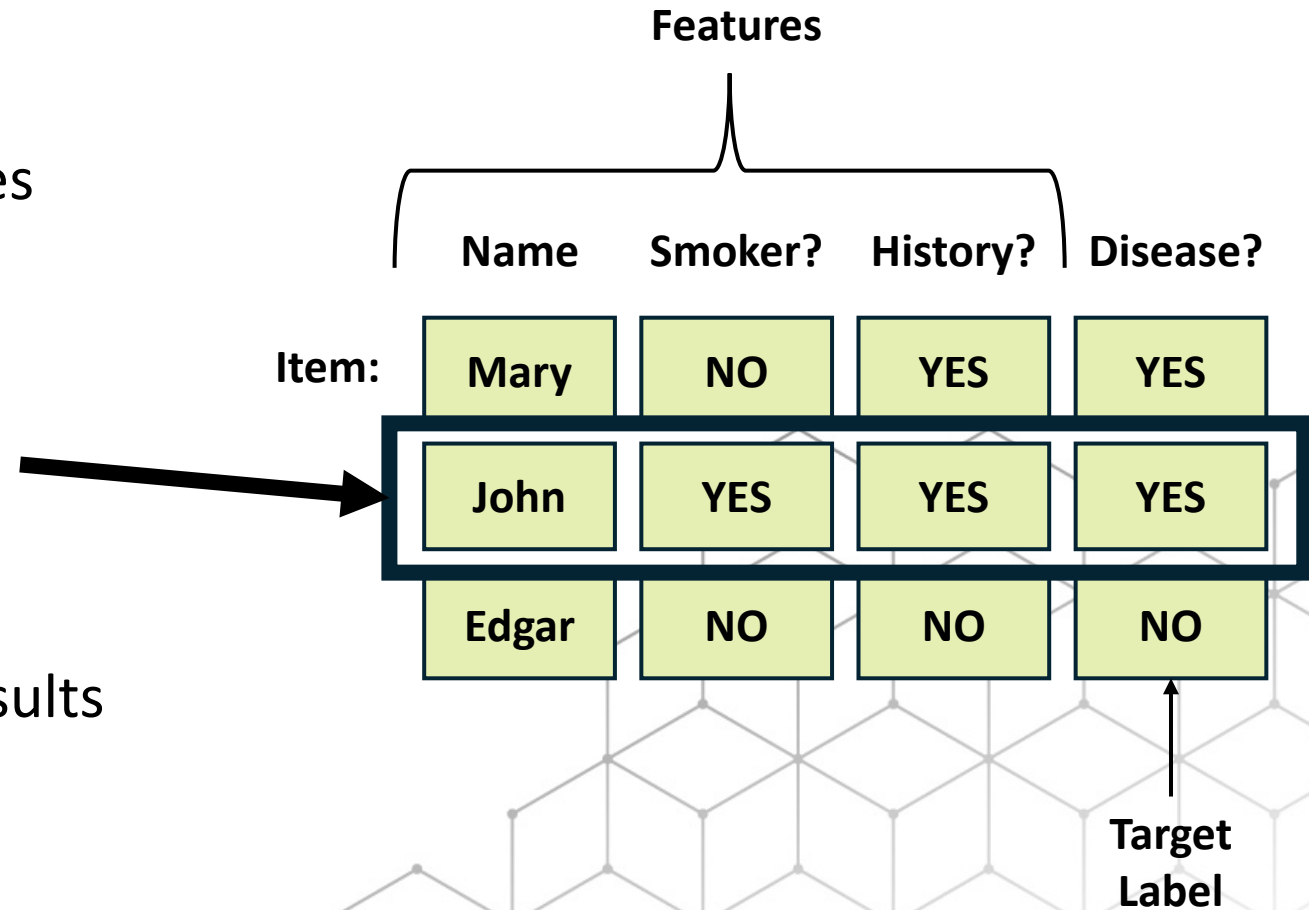
What is Big Data Analysis?

- **Process** to extract meaningful insight from big data such as hidden patterns, unknown correlations, trends.



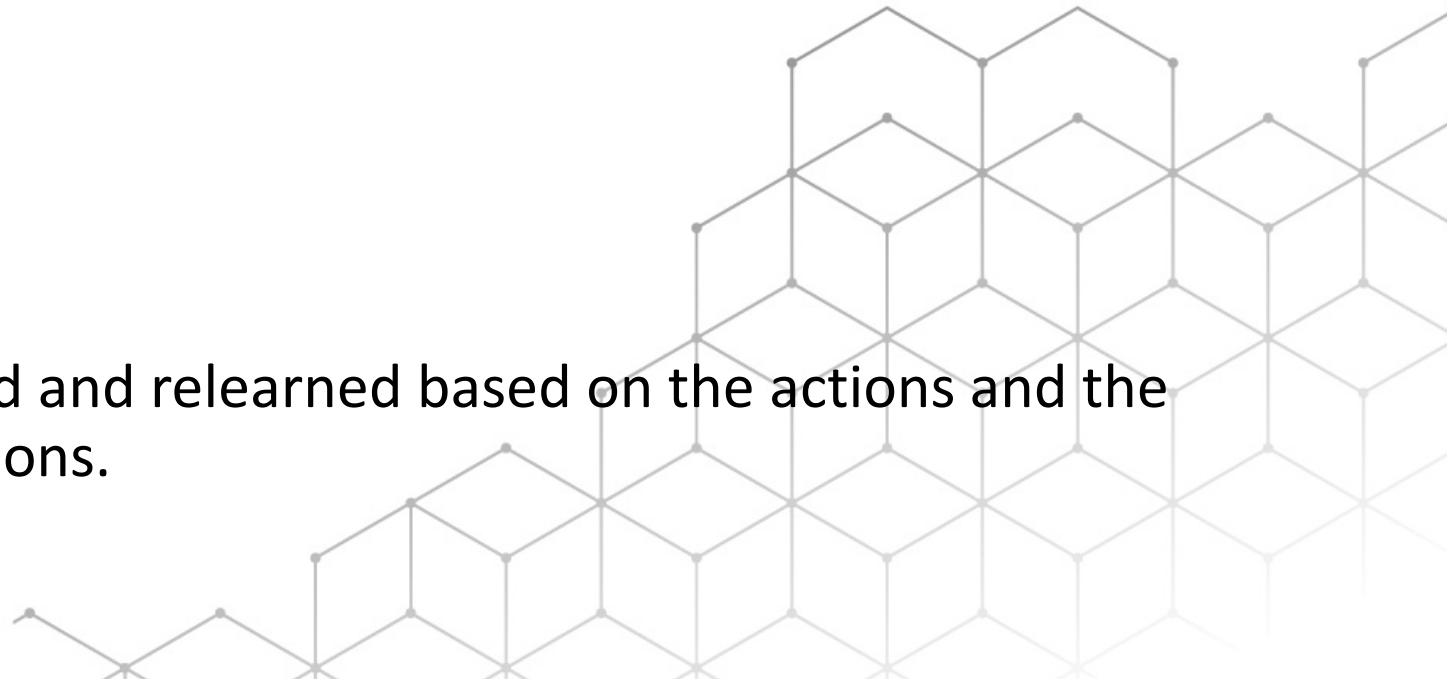
Terminology of Big Data Analysis

- Features
 - each item is described by features
- Feature vector
 - n-dimensional vector of numerical/categorical features representing some piece of data
- Labelled data
 - data with known classification results
- Model is our trained algorithm

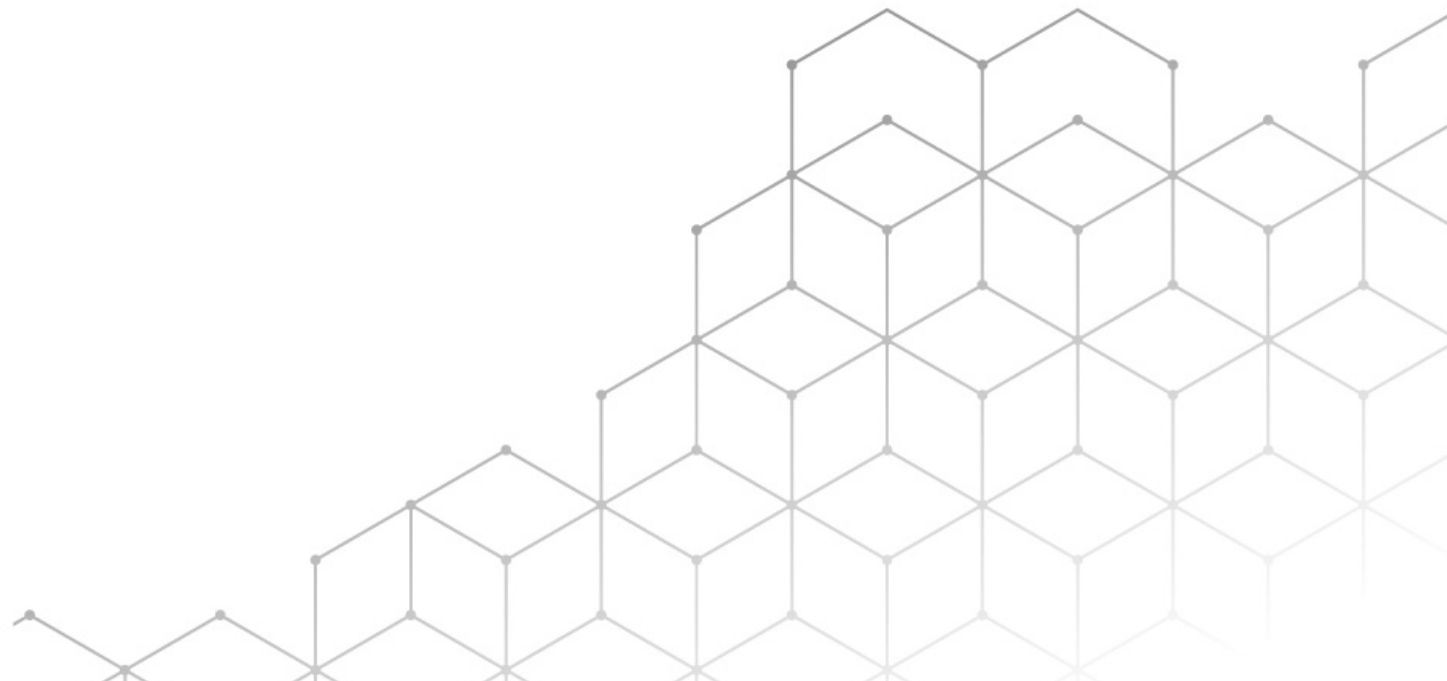
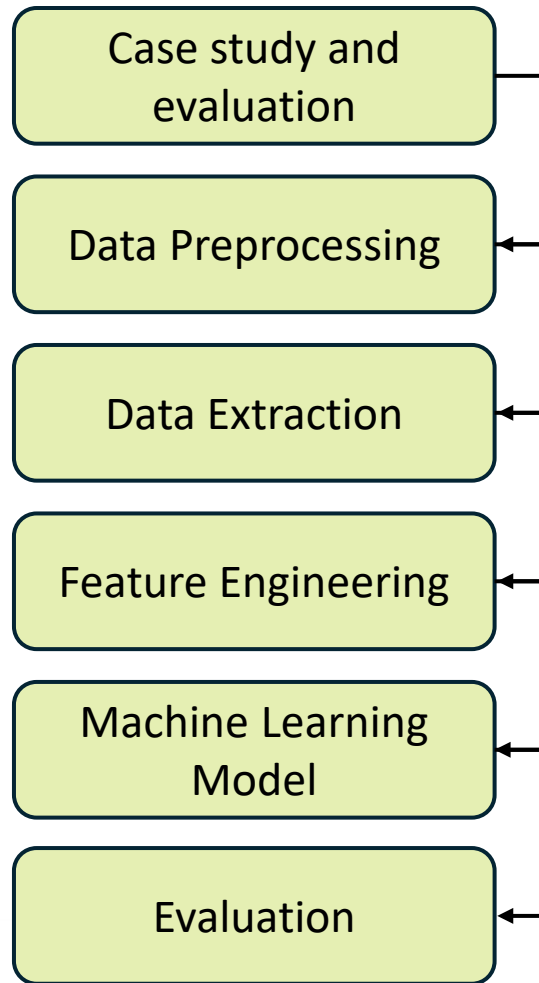


Three Types Of Learning Problems

- Supervised
 - Labelled data are available
 - Classification and Regression
- Unsupervised
 - No labelled data is available
 - Clustering, Topic Modeling
- Semi-supervised
- Reinforcement Learning
 - Model is continuously learned and relearned based on the actions and the effects/rewards from that actions.



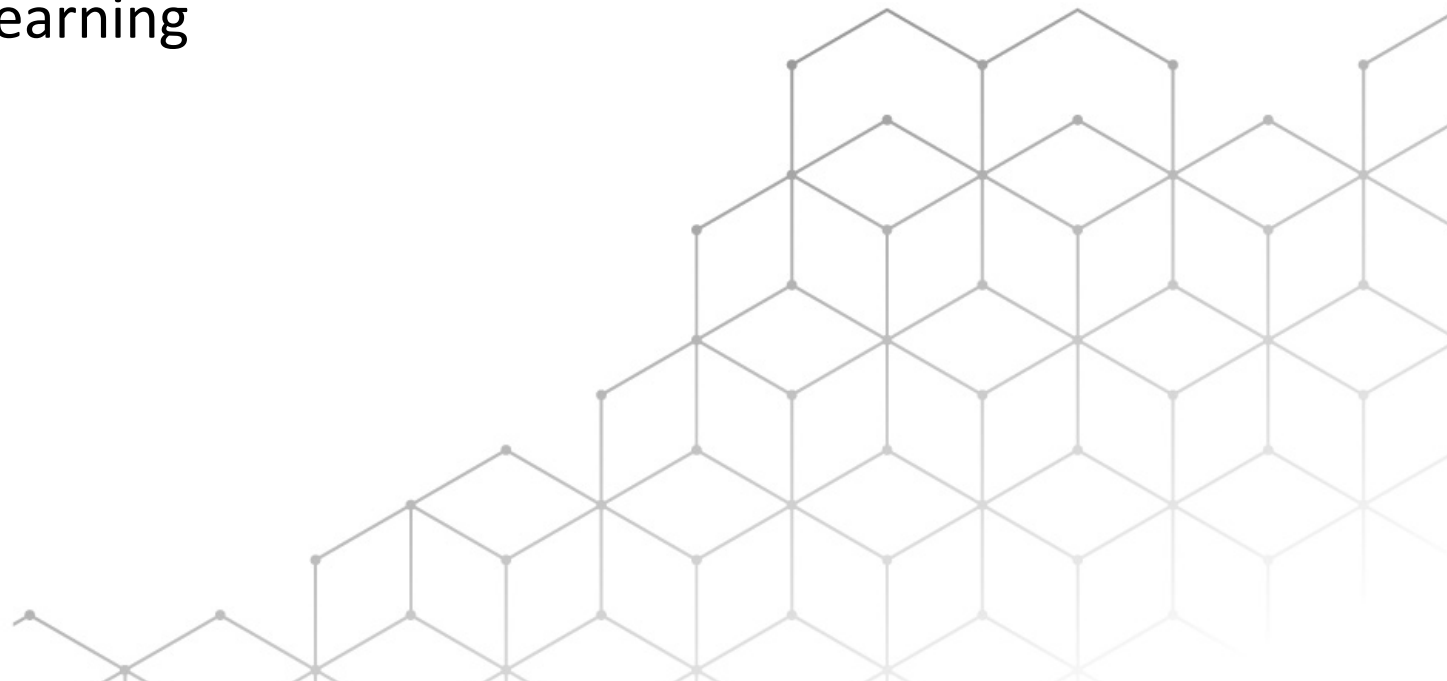
Process of Big Data Analysis



Agenda for Today

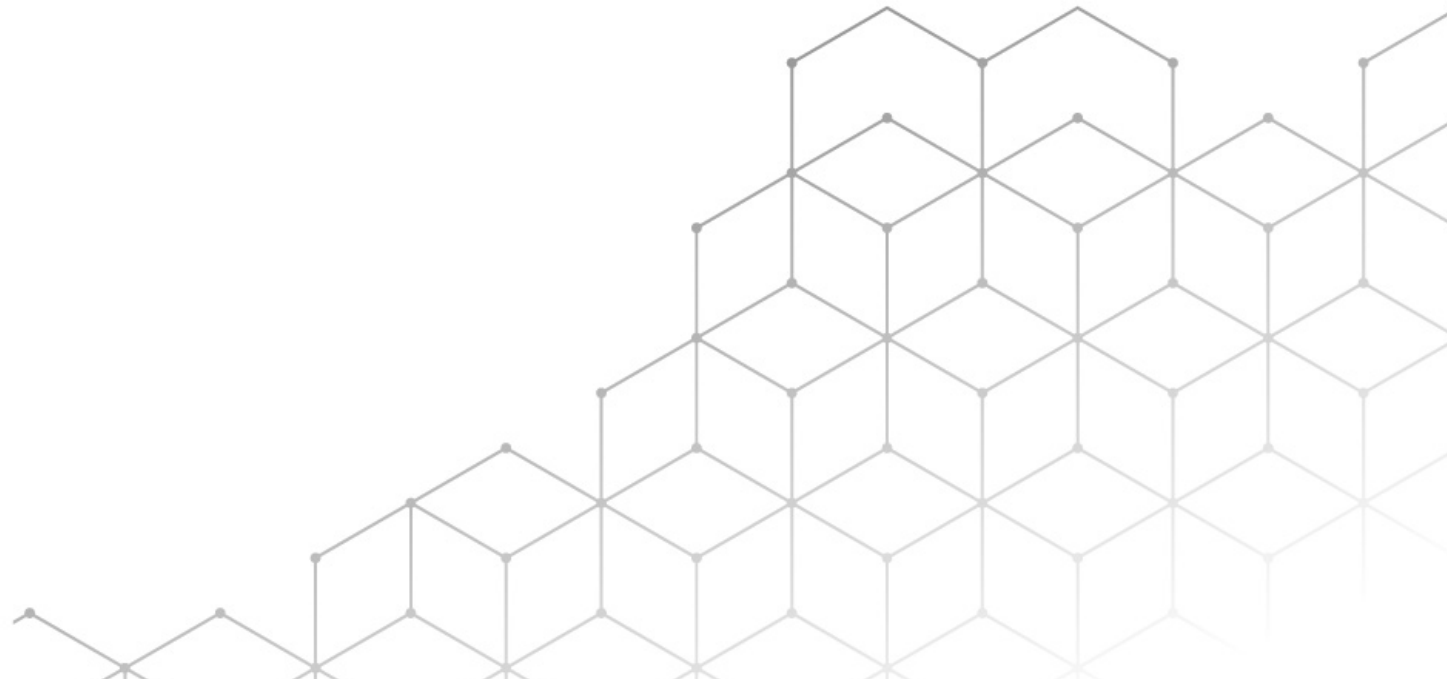


- Big Data Analysis
- Machine Learning: what is machine learning after all?
 - Why is Machine Learning Important?
 - But What is Machine Learning?
 - Supervised vs Unsupervised Learning
- Spark MLlib



Importance of Machine Learning

- Machine Learning (ML) is important because it creates models (systems) that learn from data and can make predictions without being explicitly programmed.

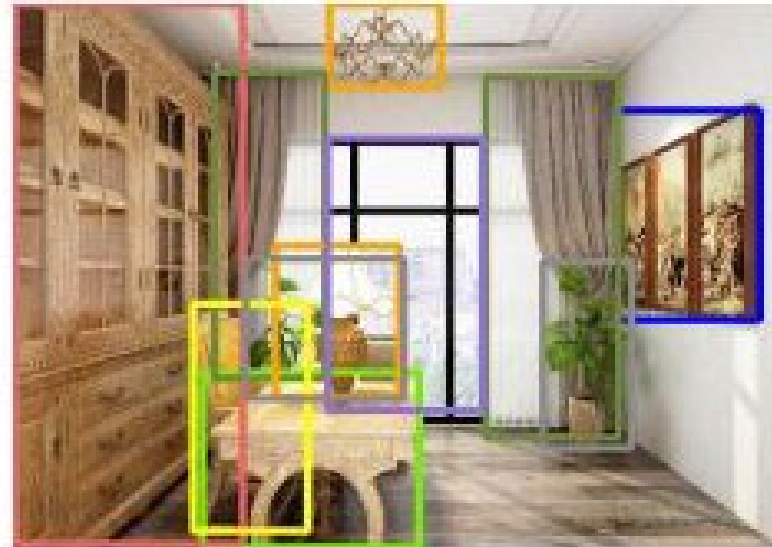


Examples of Machine Learning

- Detecting cats and dogs (or other objects) in pictures and videos
- We can now process speech much better than before
- But some of the best:
 - <https://www.youtube.com/shorts/bwgl1vkSe28>
 - [Quick, Draw!](#)
 - <https://www.md.ai/>



Furniture Detection



Dog & Cat Detection & Segmentation data with purpose

Classification



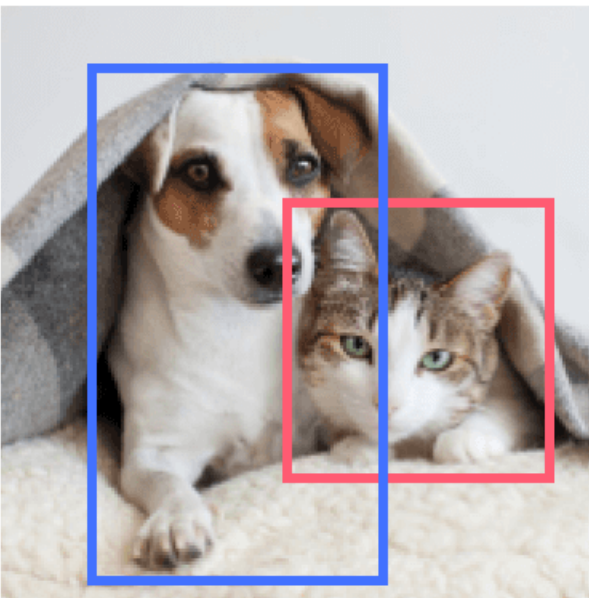
Cat

**Classification
+ Localization**



Cat

Object Detection



Cat, Dog

Instance Segmentation



Cat, Dog

Single object

Multiple objects

NOVA

IMS

Information
Management
School

Advances even in Arts

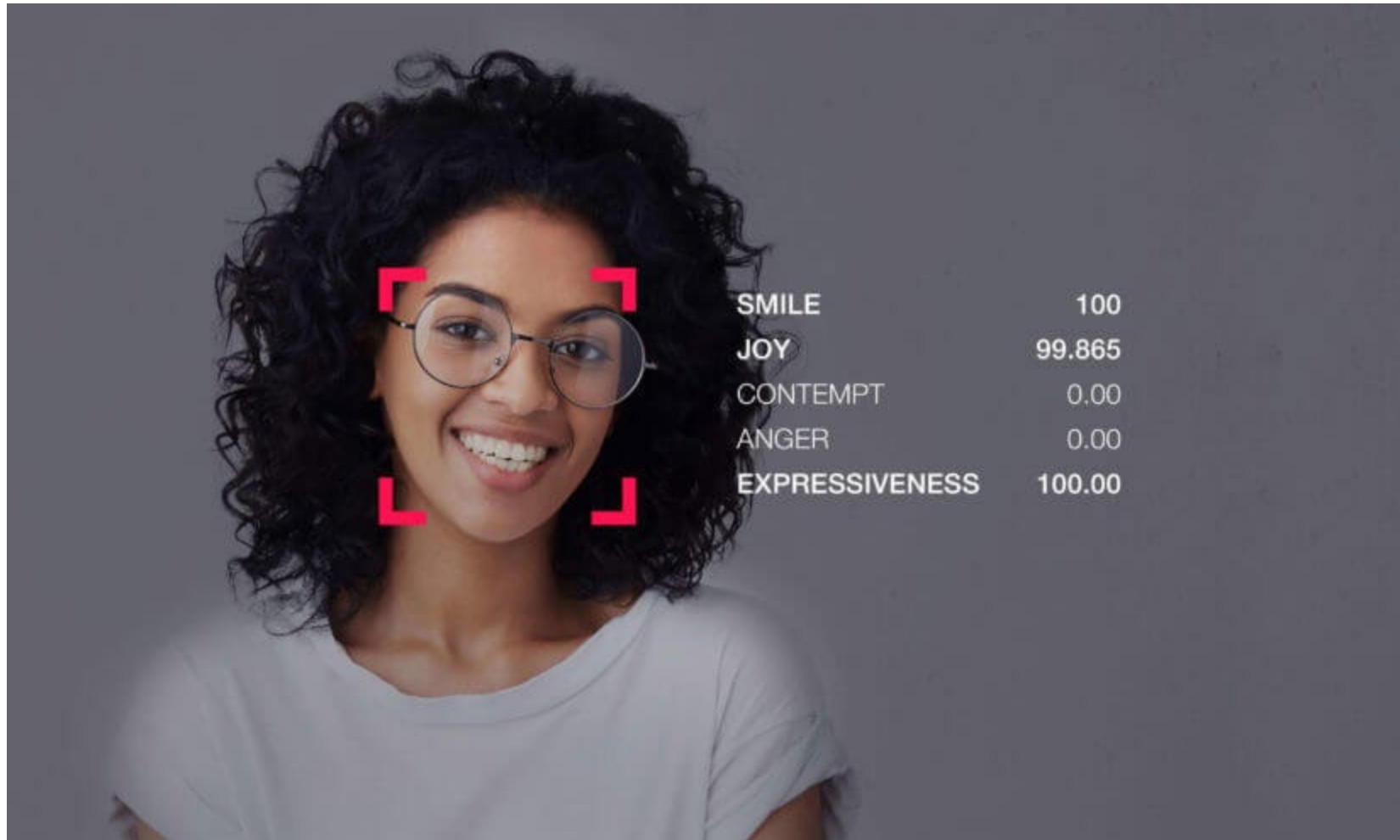


NOVA

IMS

Information
Management
School

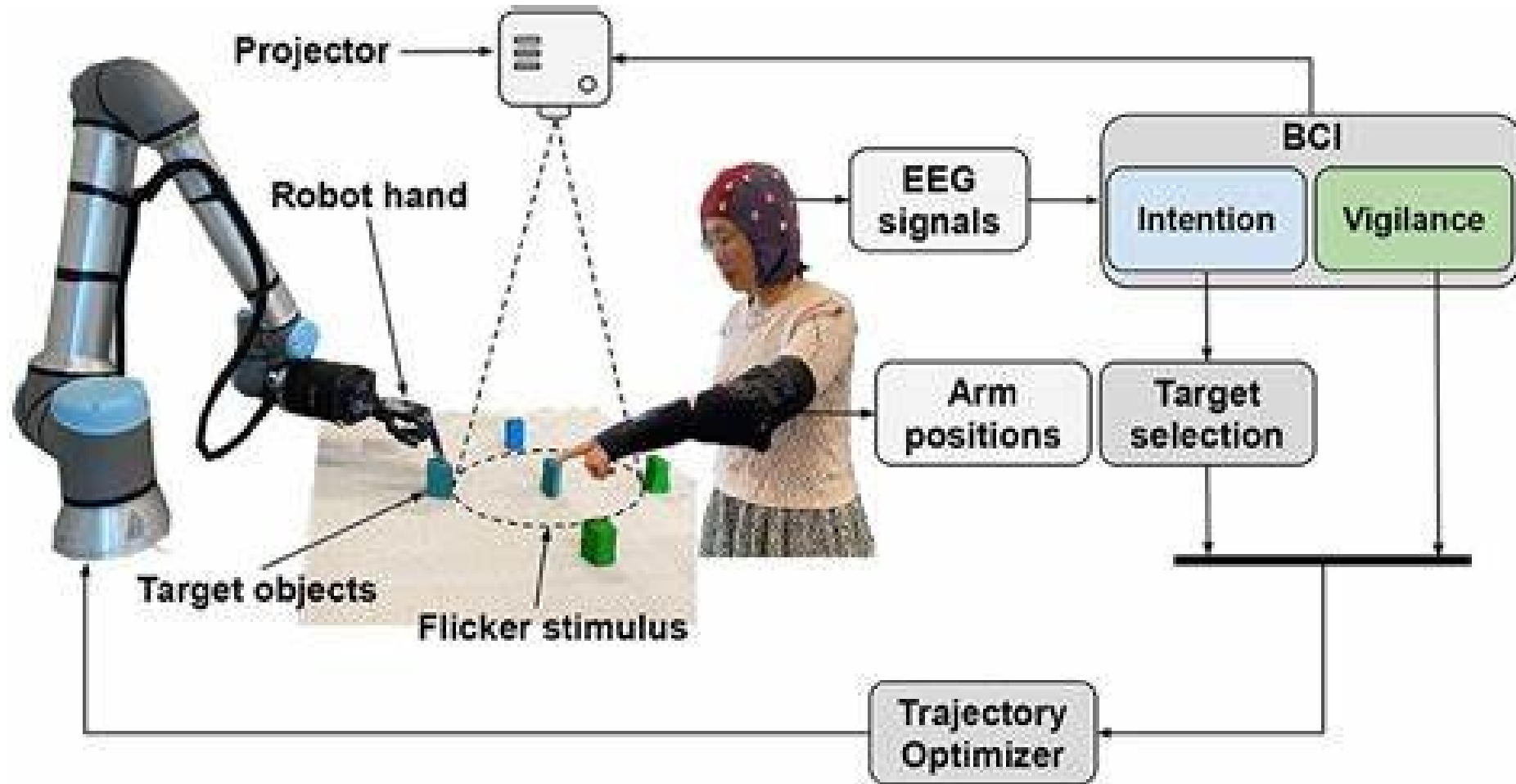
Advances in Emotions



Brain Computer Interface (BCI)

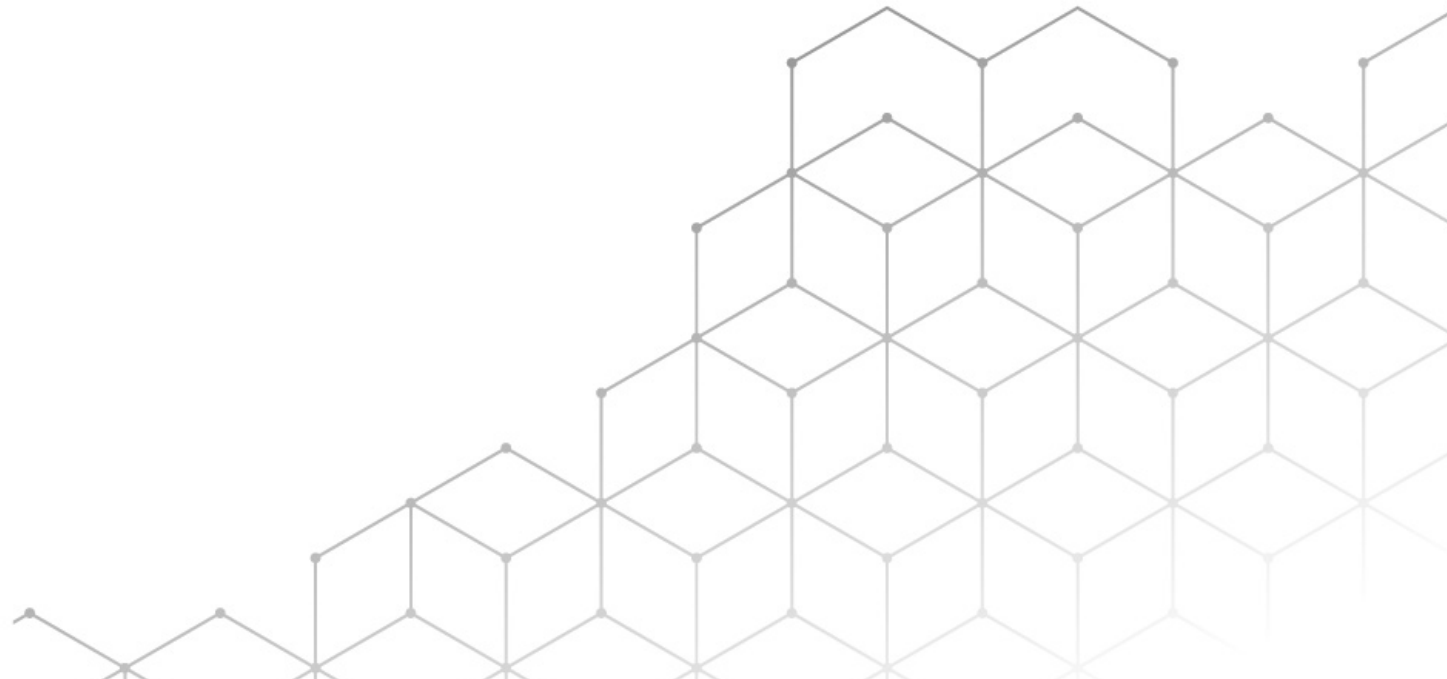


Coordinating Human-Robot Collab



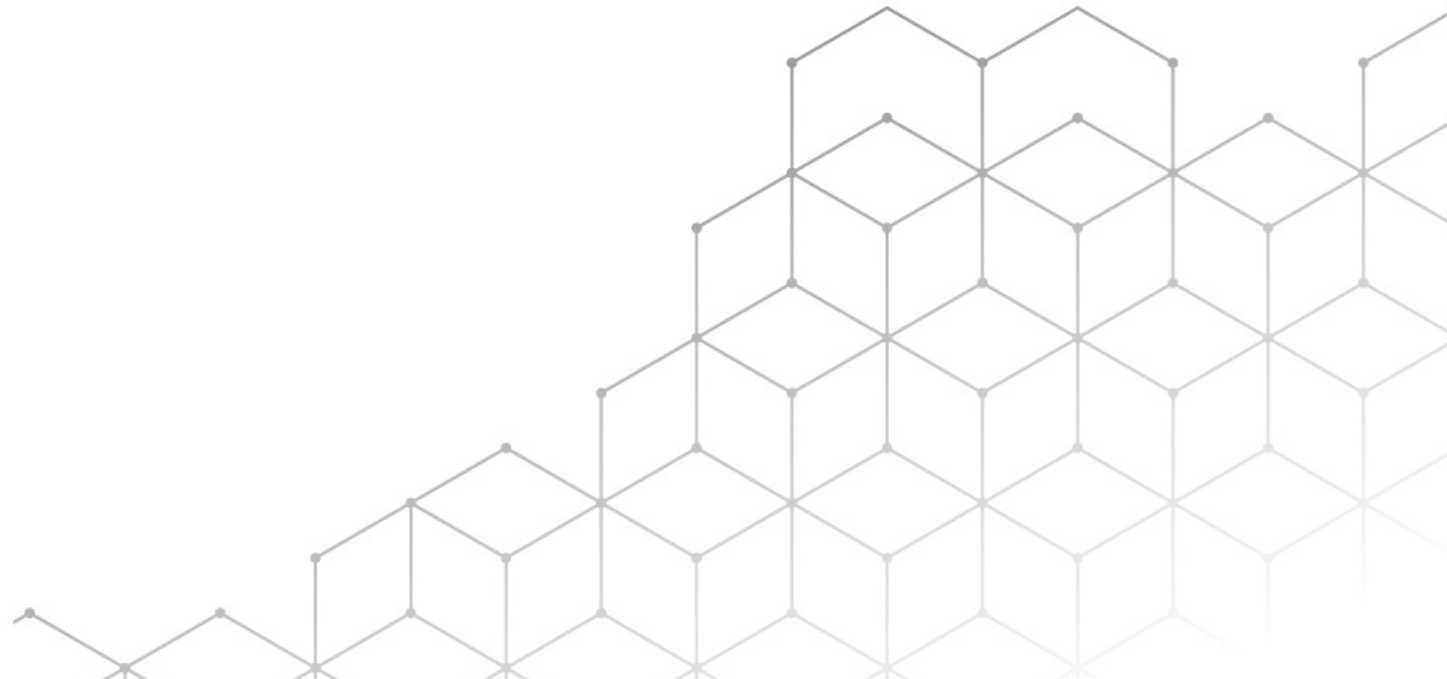
Poetry and translation too...

- <https://poemgenerator.io/>
- https://youtu.be/8ETF92WMz_0



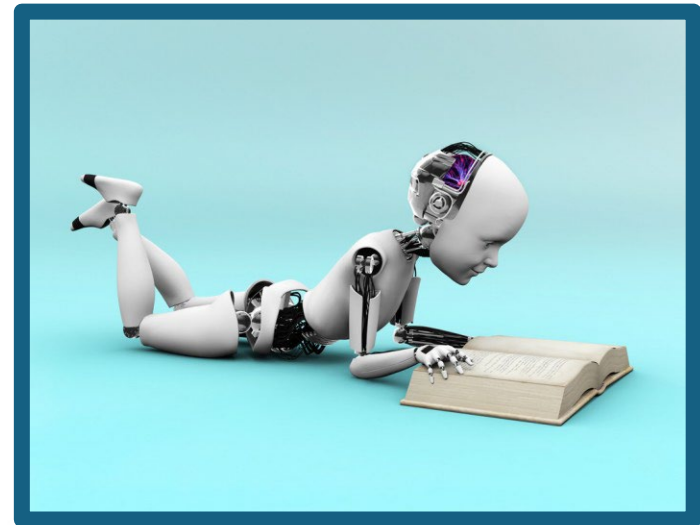
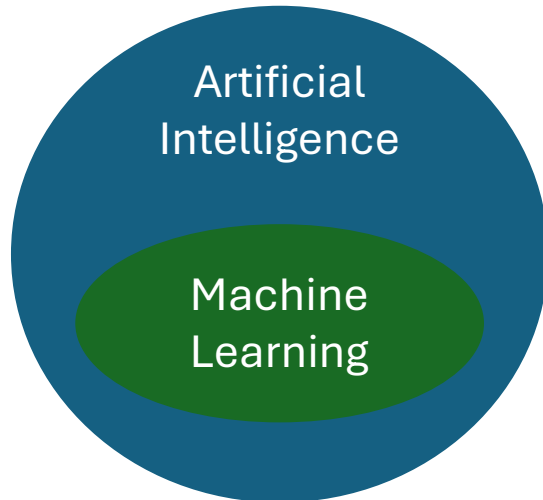
Deep Fakes and Ethics

- <https://youtu.be/NxSOG1aNeBo>



But... what is Machine Learning?

- Machine learning is a branch of Artificial Intelligence (AI) that enables computers to learn patterns from data and make decisions or predictions without being **explicitly** programmed.



Typical Computer Program

- # A simple Python program

```
# Ask the user for their name
```

```
name = input("What is your name? ")
```

```
# Ask the user for their age
```

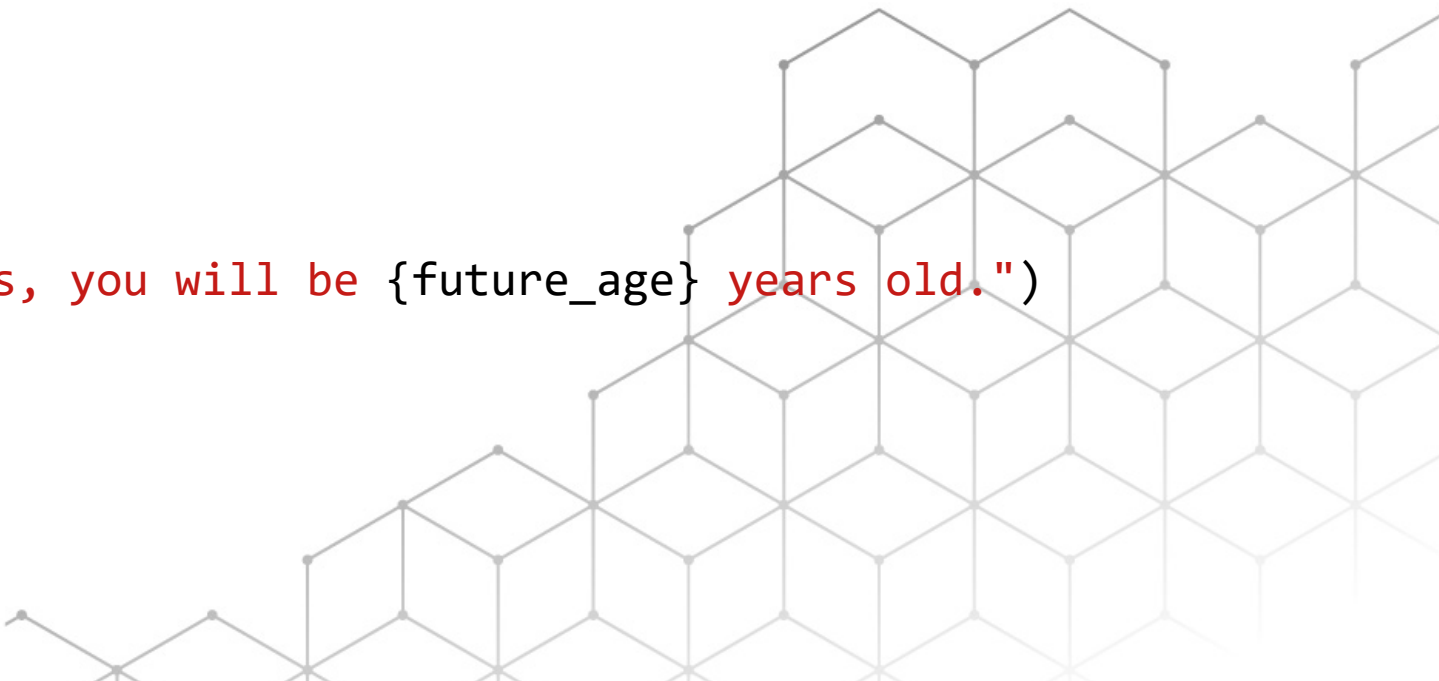
```
age = int(input("How old are you? "))
```

```
# Calculate future age
```

```
future_age = age + 10
```

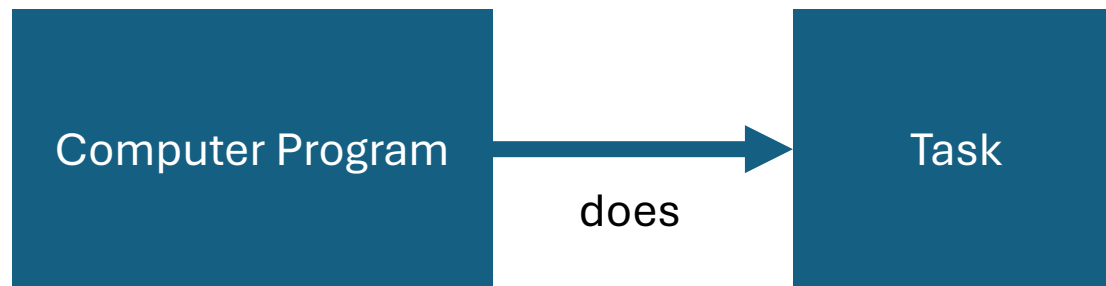
```
# Display the result
```

```
print(f"Hello, {name}! In 10 years, you will be {future_age} years old.")
```



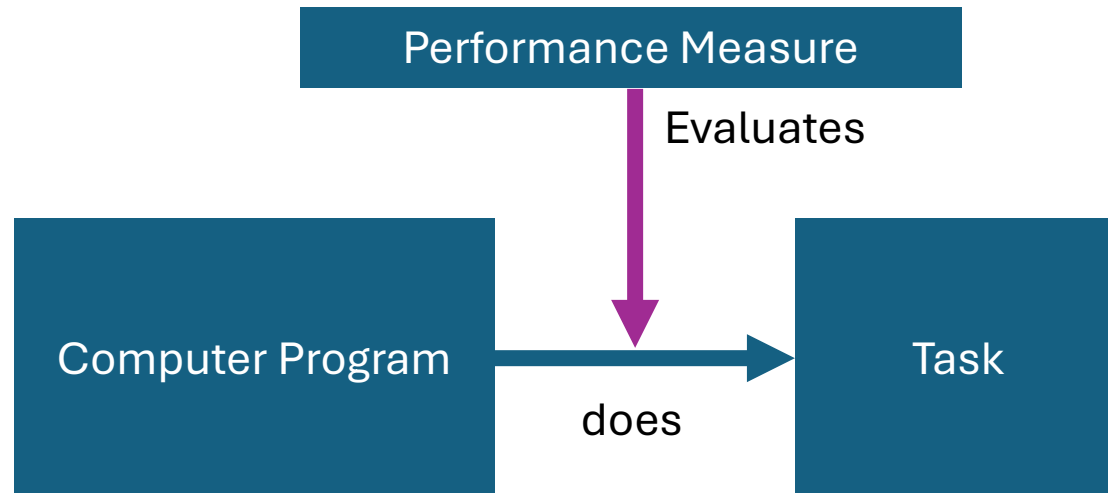
Formal definition

- A computer program is said to learn from experience E with respect to some task T and some performance measure P if its performance on task T , as measured by P , improves with experience E .



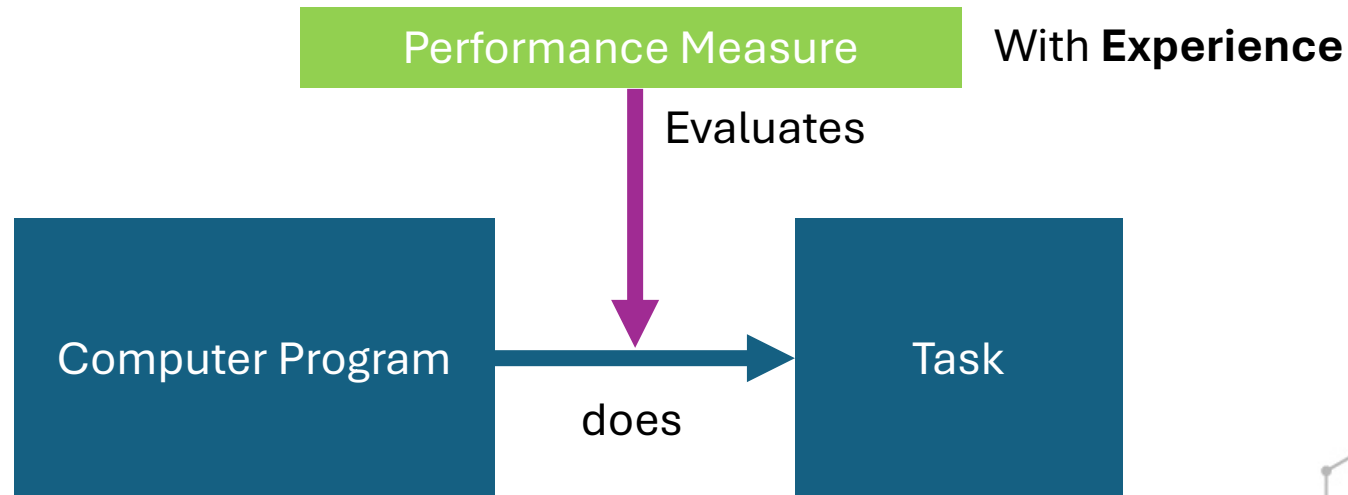
Formal definition

- A computer program is said to learn from experience E with respect to some task T and some performance measure P if its performance on task T , as measured by P , improves with experience E .



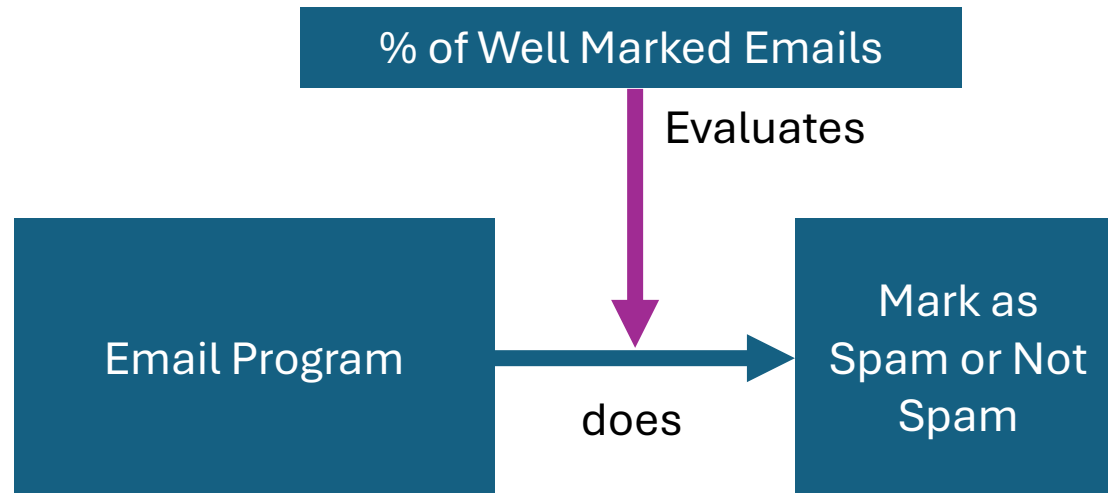
Formal definition

- A computer program is said to learn from experience E with respect to some task T and some performance measure P if its performance on task T , as measured by P , improves with experience E .



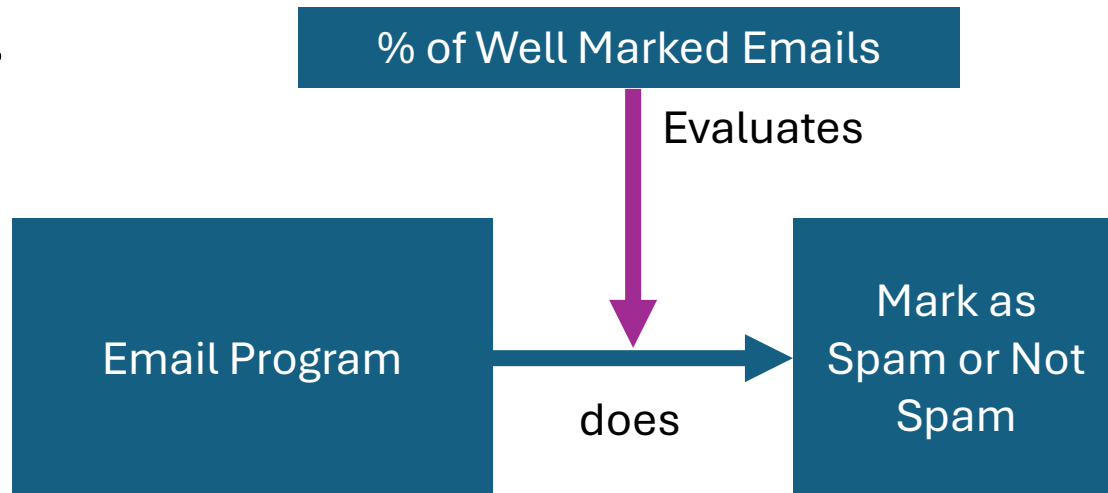
Example of learning task

- Suppose your email program watches which emails you do or do not mark as spam, and based on that learns how to better filter spam. What is the task T in this setting?



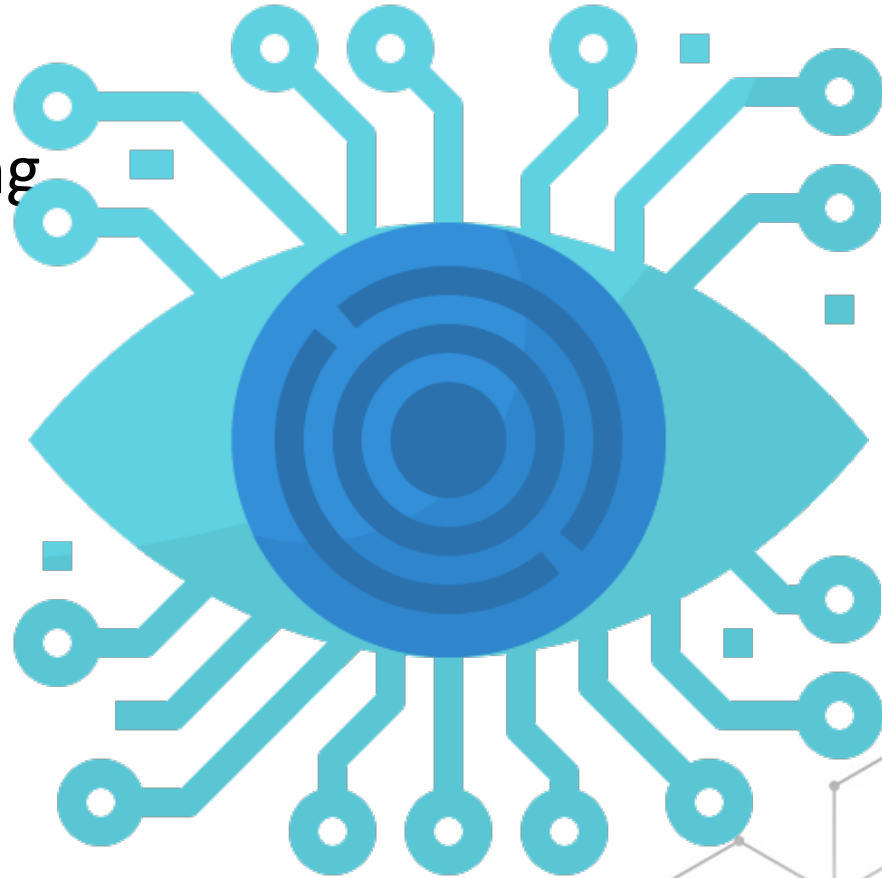
Solution for previous example

- T := Classifying emails as spam or not spam.
- E := Watching you label emails as spam or not spam.
- P := The number (or fraction) of emails correctly classified as spam/not spam.



Types of Problems and tasks

- Supervised learning
- Unsupervised learning

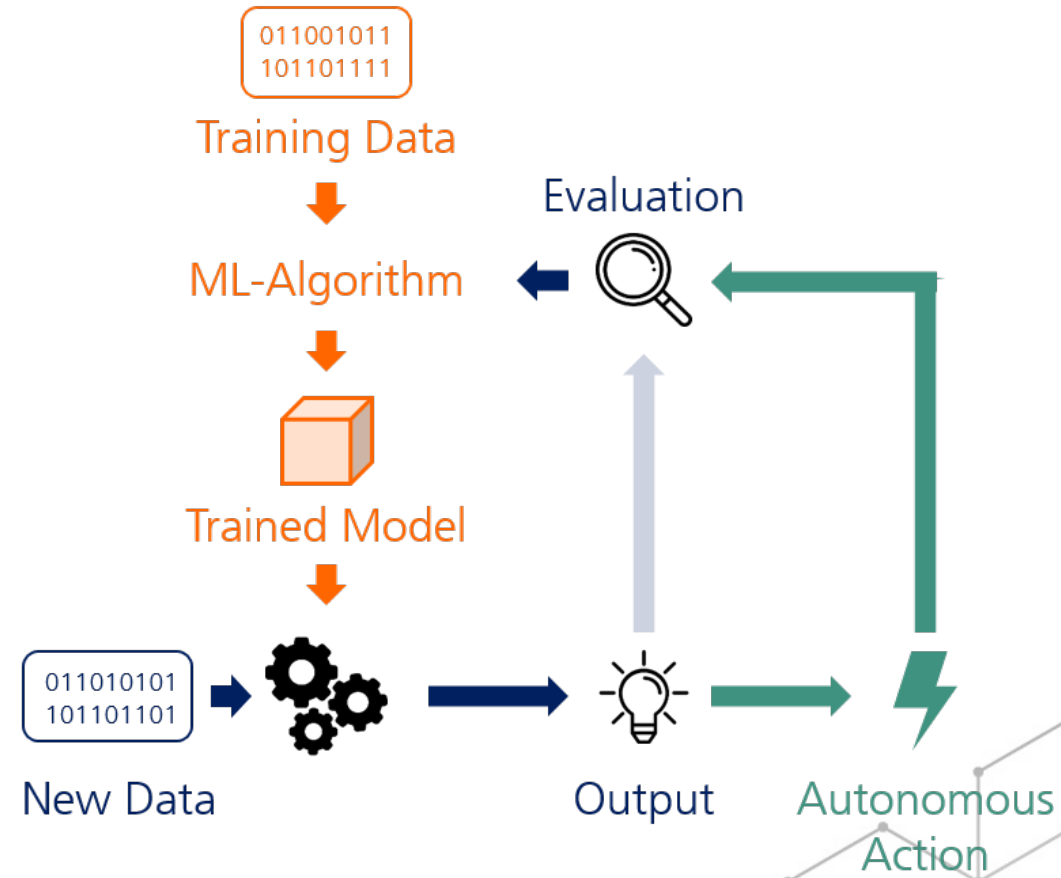


Supervised Learning

- In supervised learning, we are given a data set and already know what our correct output should look like, having the idea that there is a relationship between the input and the output.
- Learning from examples
- E.g.: Spam Detection, Face Detection, Recommendations

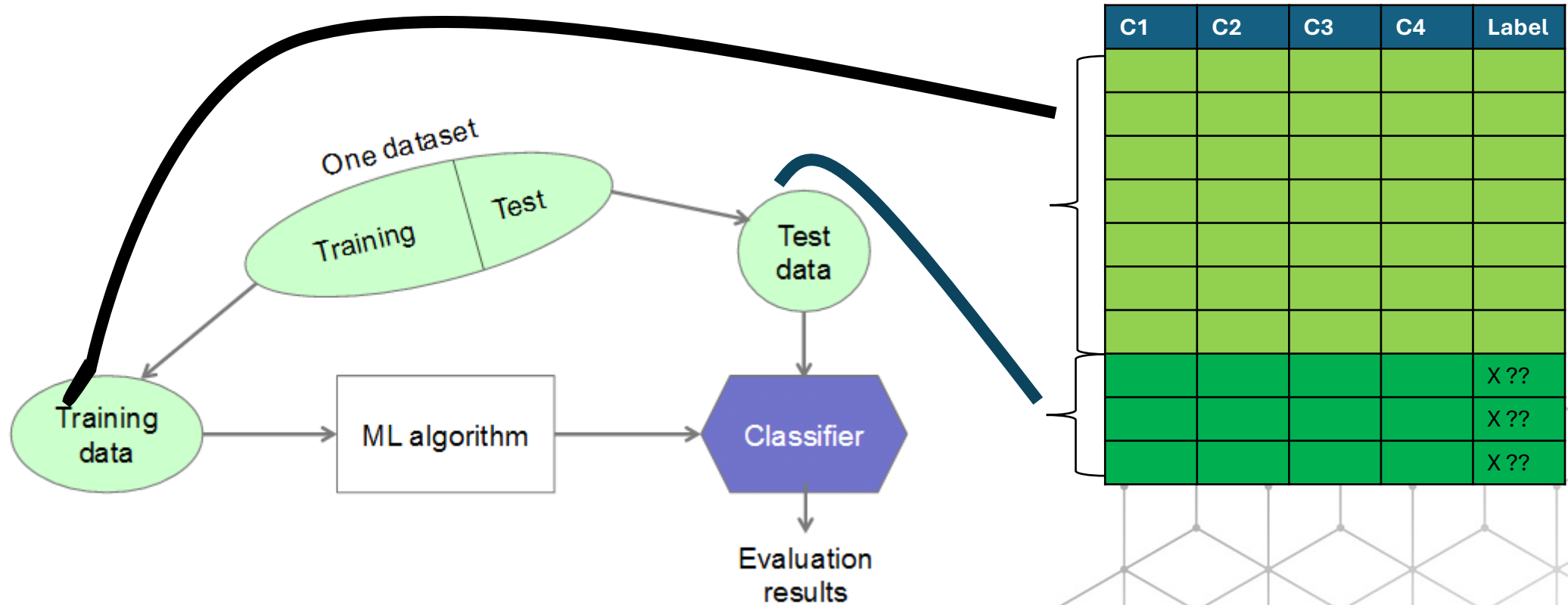


New Data

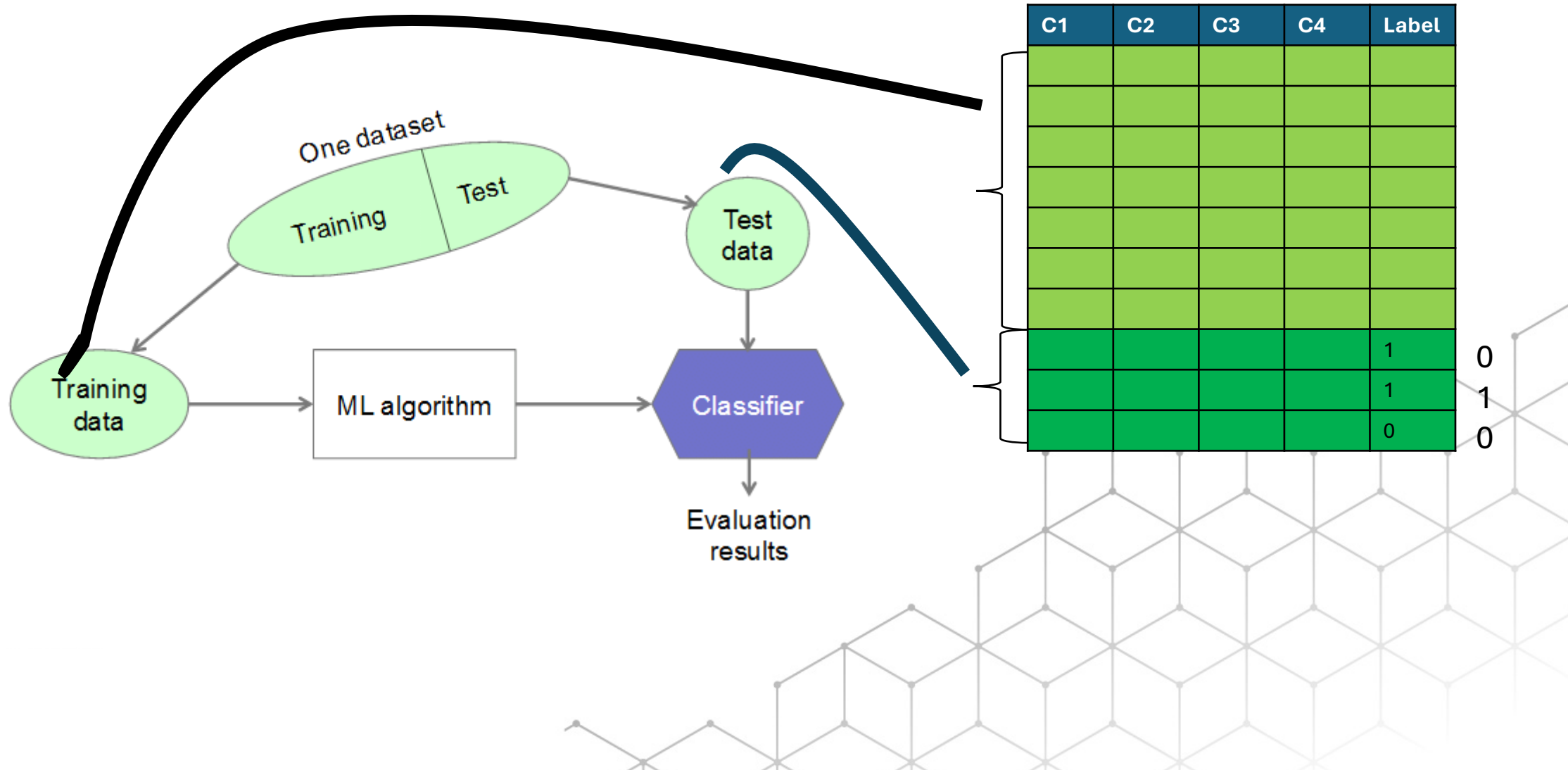


	C1	C2	C3	C4	Label
A					
B					X ??
					X ??
					X ??

How do we divide the DATA?

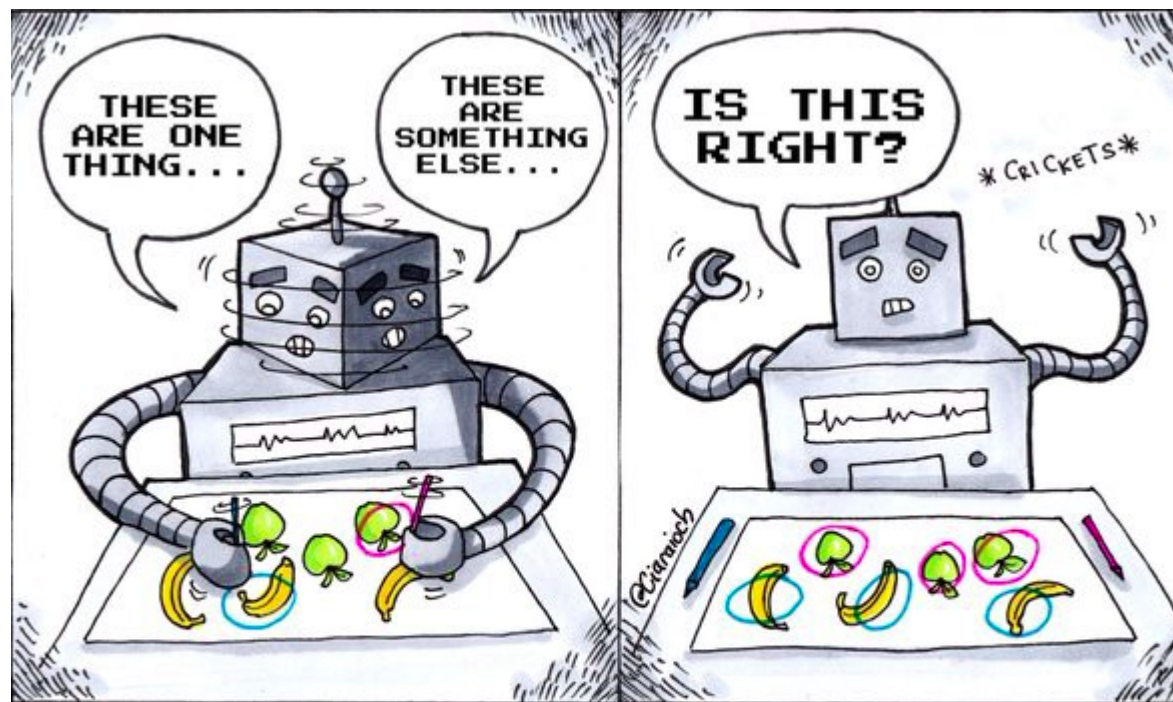


How do we divide the DATA?



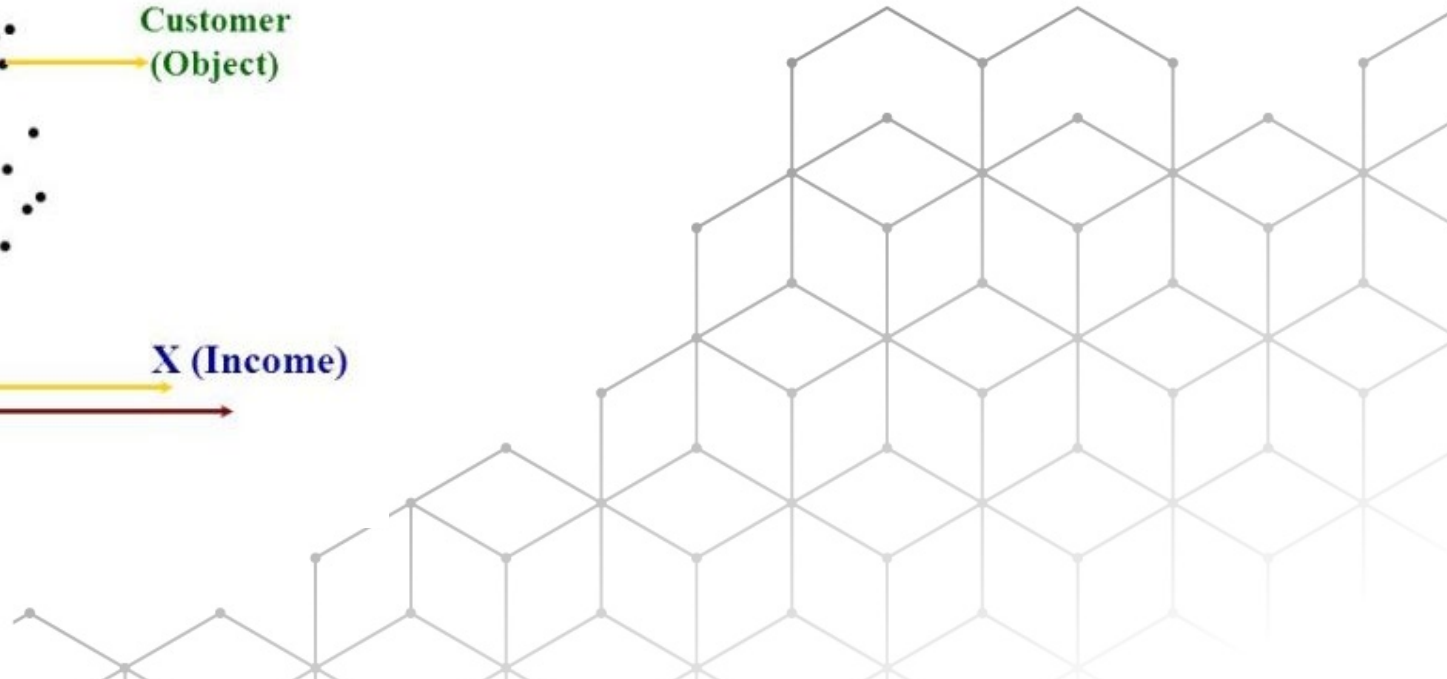
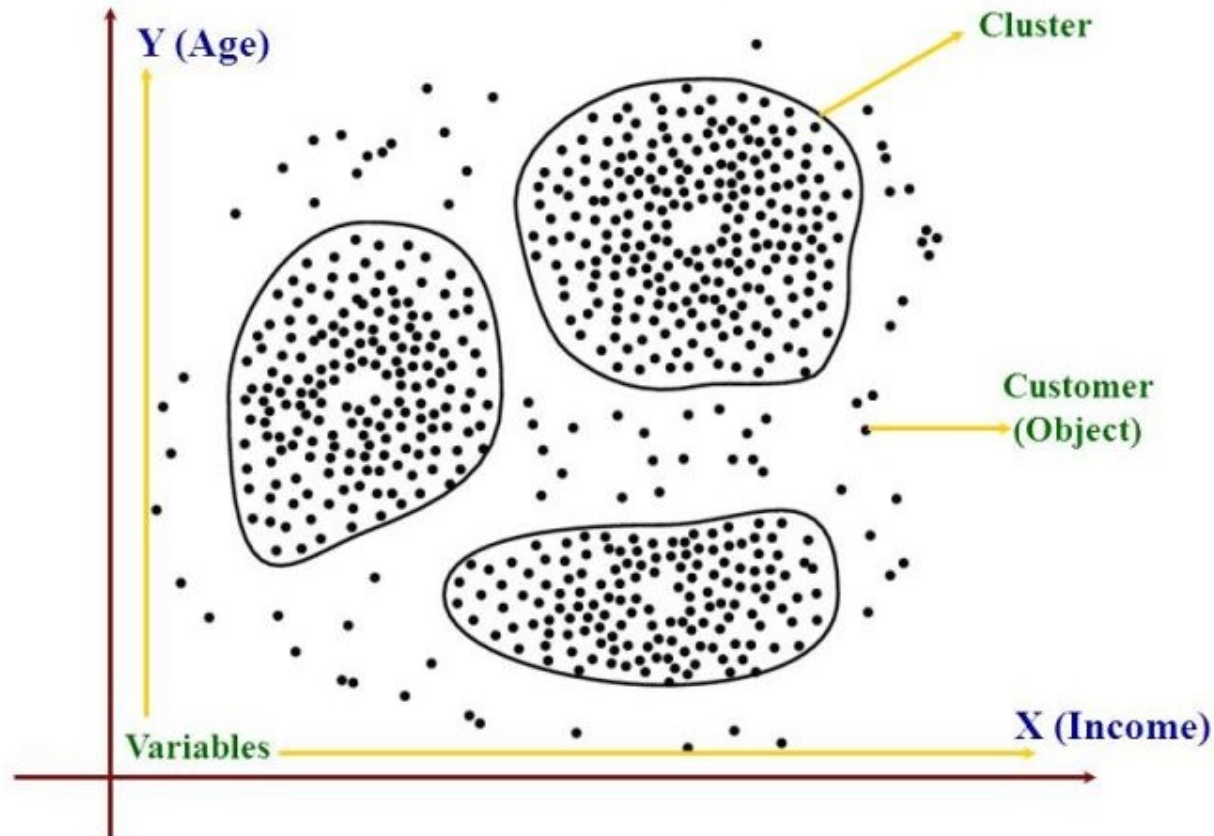
Unsupervised Learning

- Unsupervised algorithms do not need to be trained with desired outcome data. Instead, they use an iterative approach called deep learning to review data and arrive at conclusions.
- Eg: Grouping that are somehow similar or related by different variables.

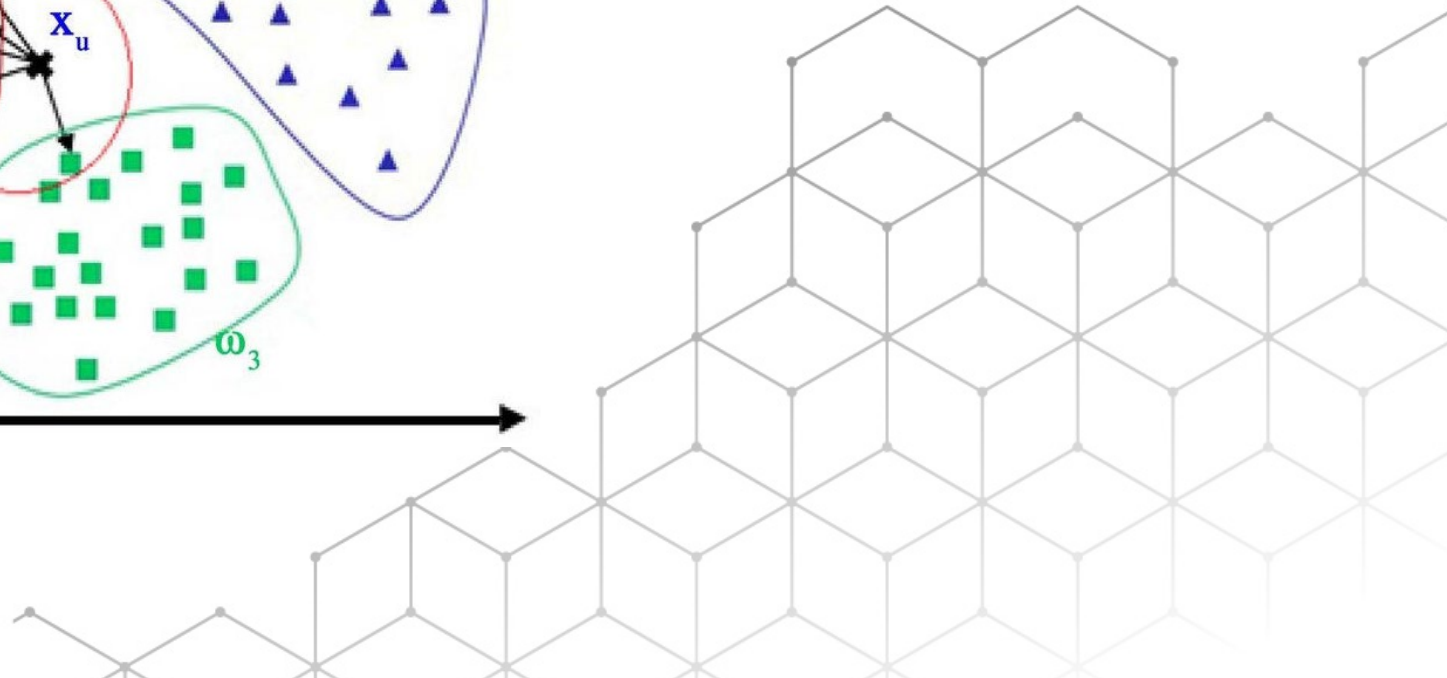
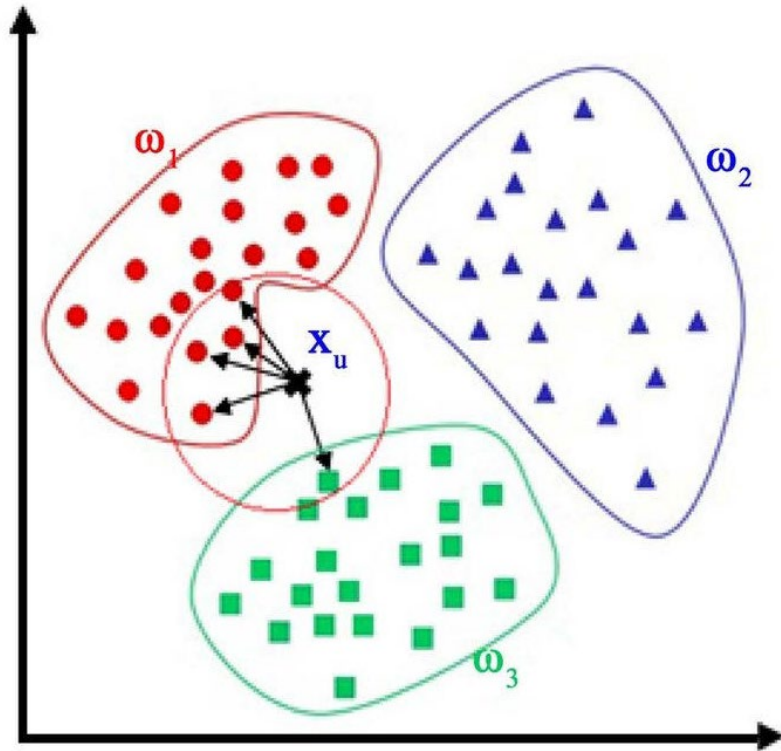


Unsupervised Learning

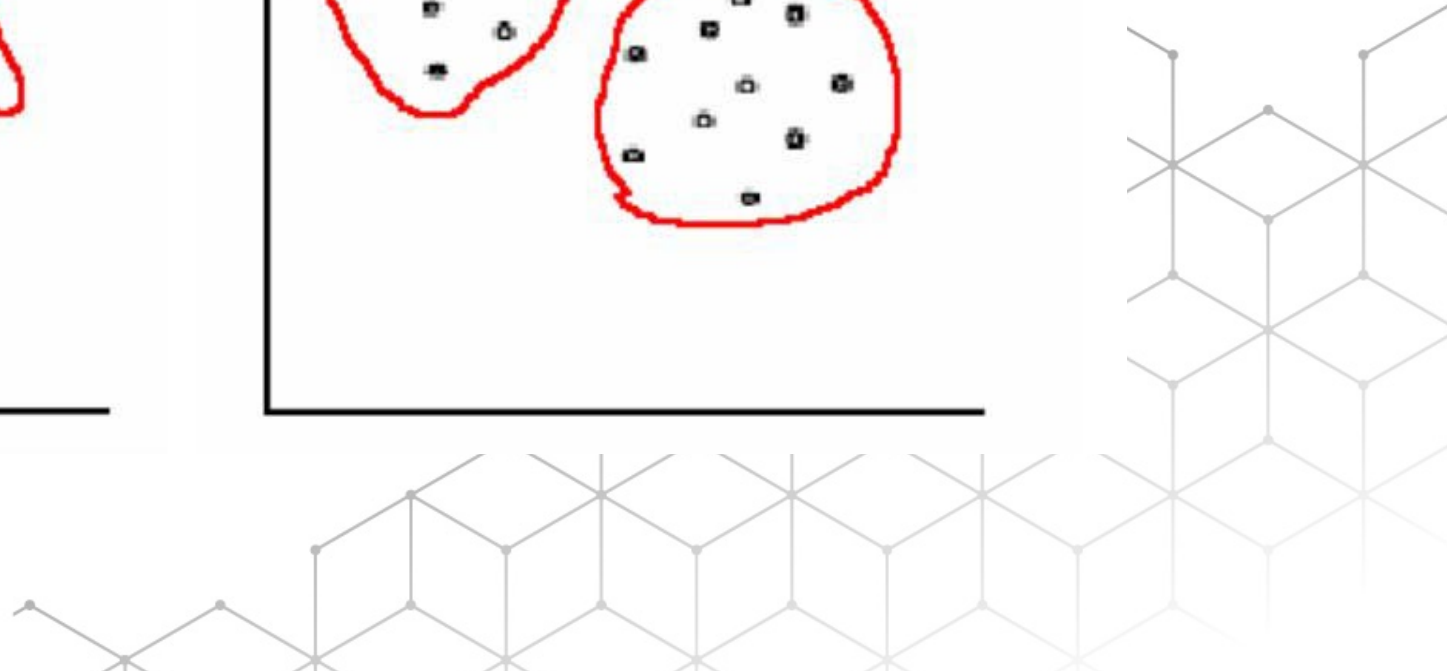
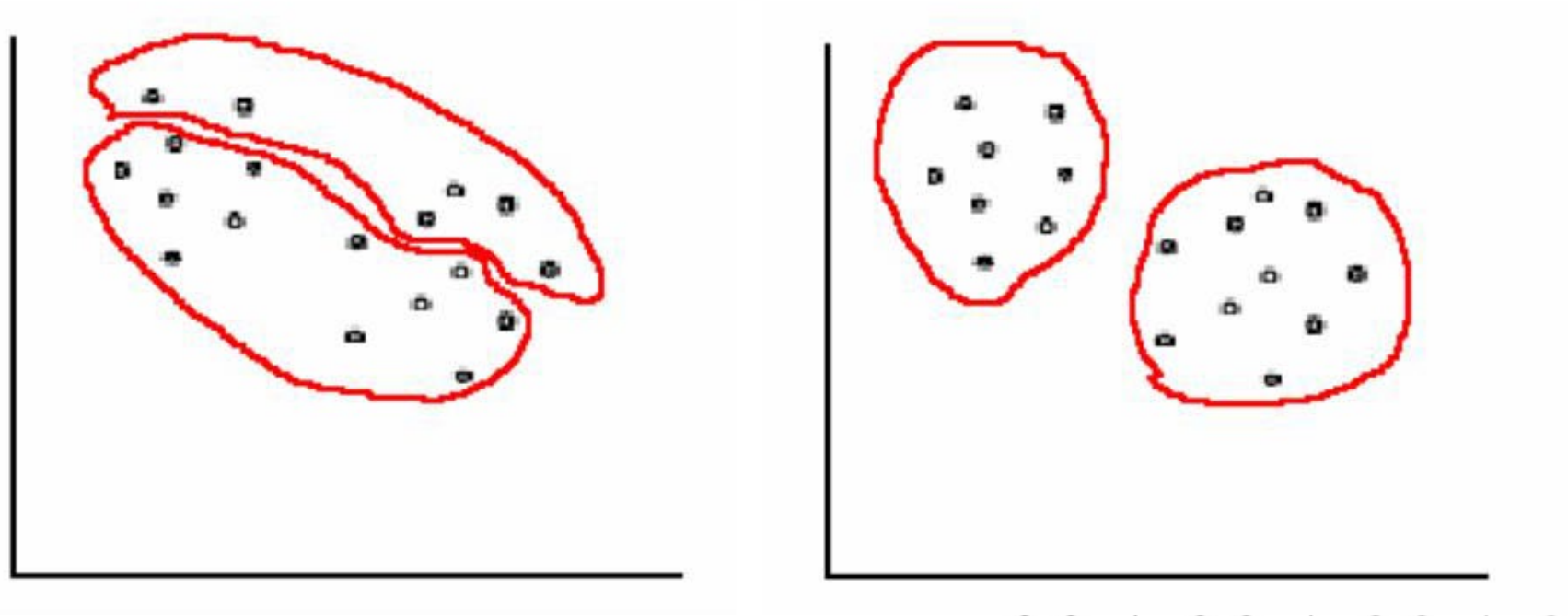
HOW TO DO UNSUPERVISED LEARNING



How to calculate Membership?

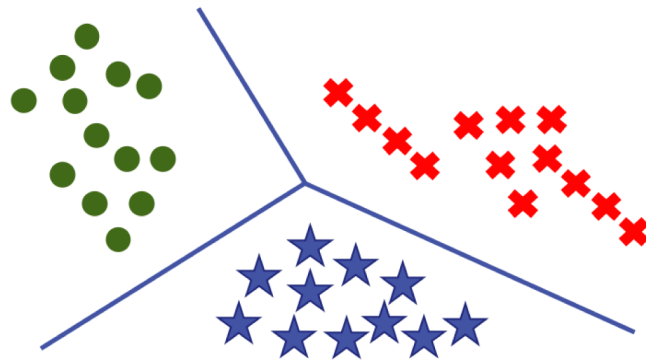


BAD CLUSTERING VS GOOD CLUSTERING

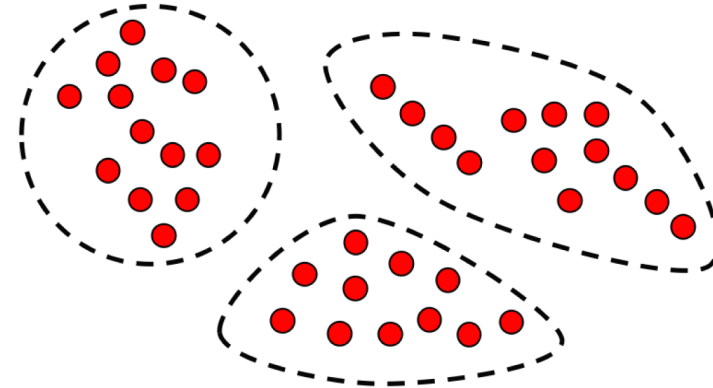


[illegible]

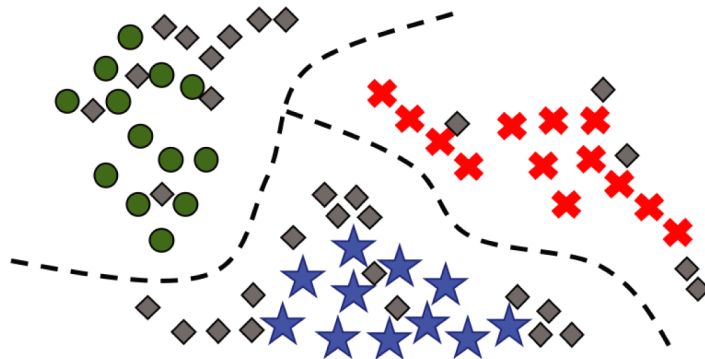
LET US COMPLICATE...



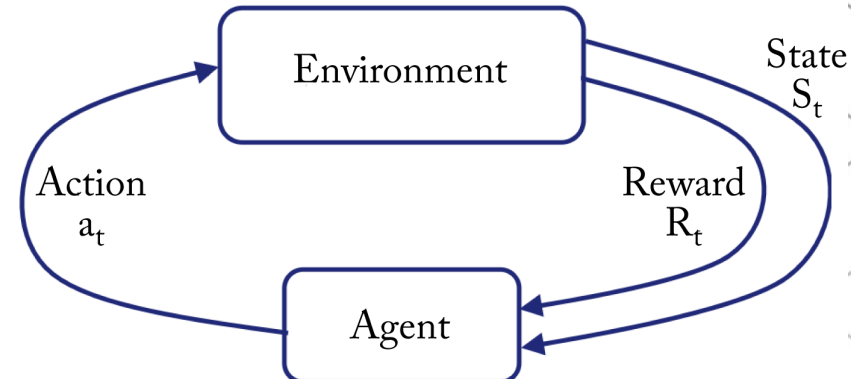
(a) Supervised Learning



(b) Unsupervised Learning



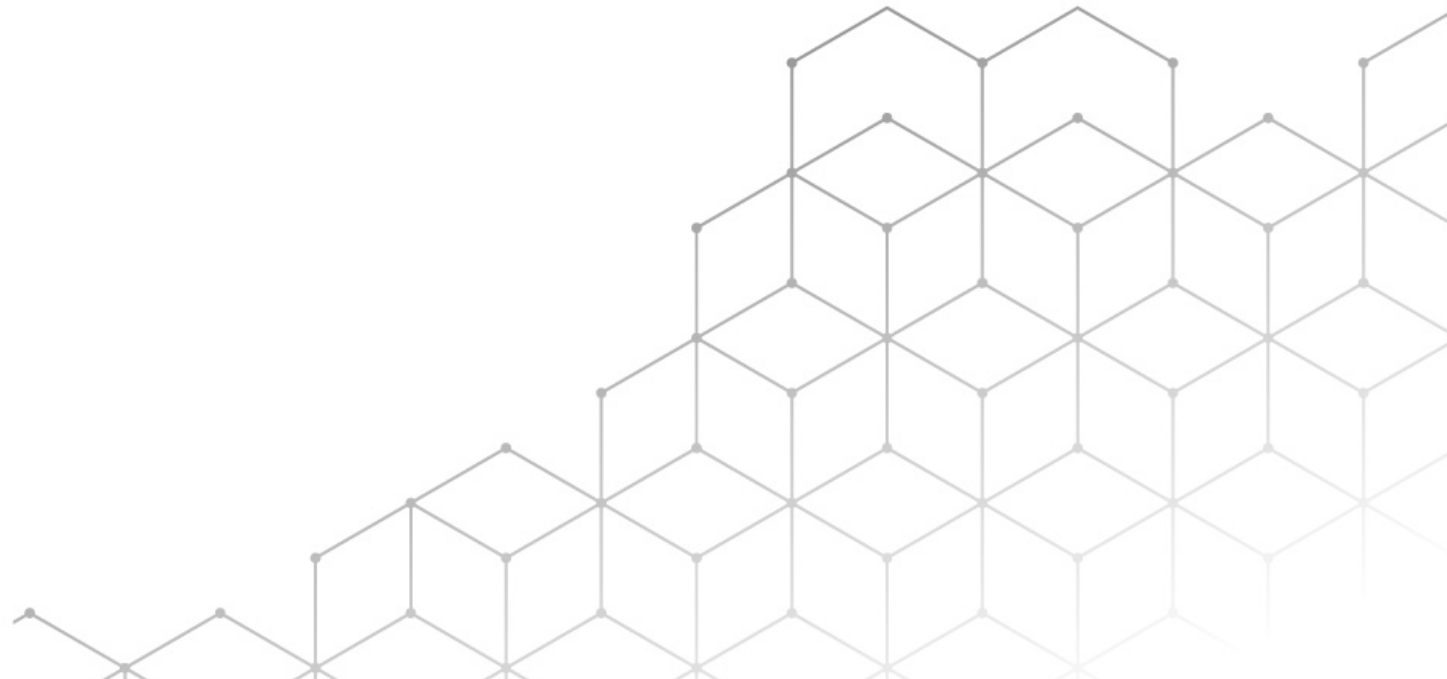
(c) Semi-Supervised Learning



(d) Reinforcement Learning

End of sLIDES

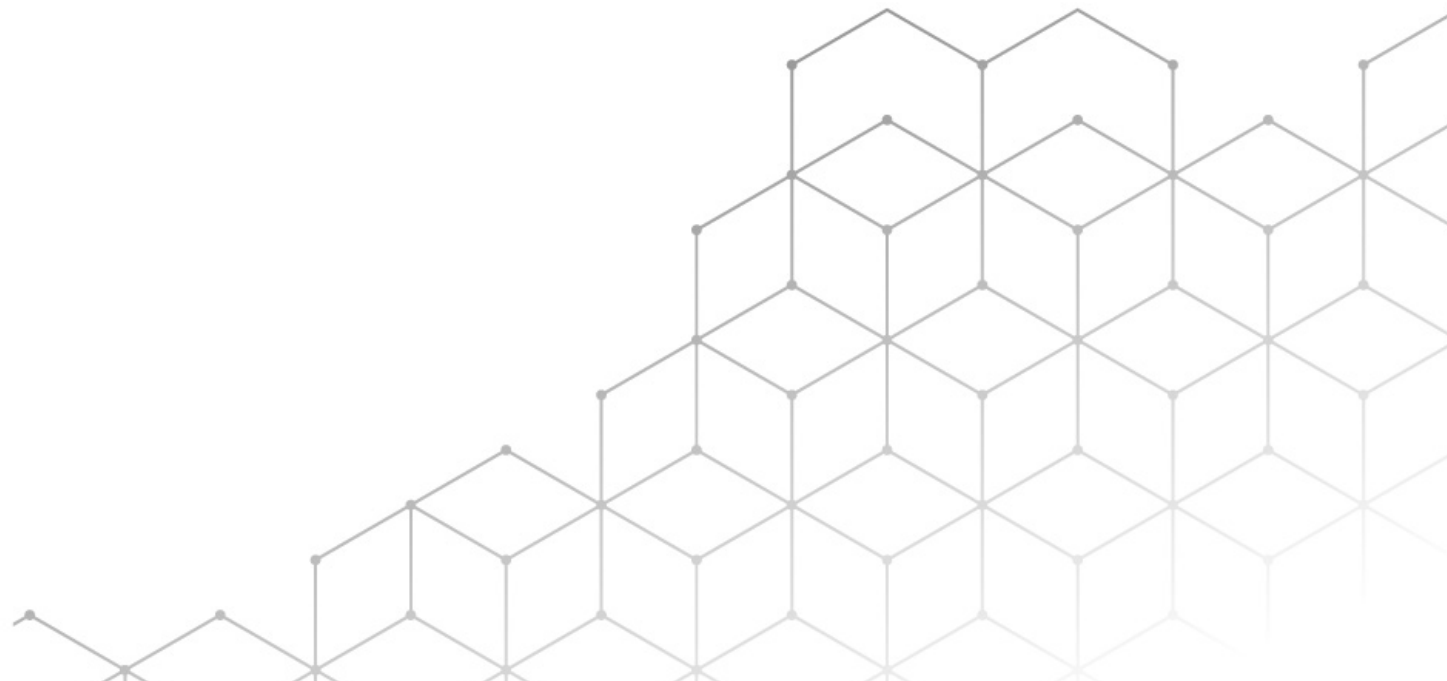
Thanks for Staying with Us



Agenda for Today

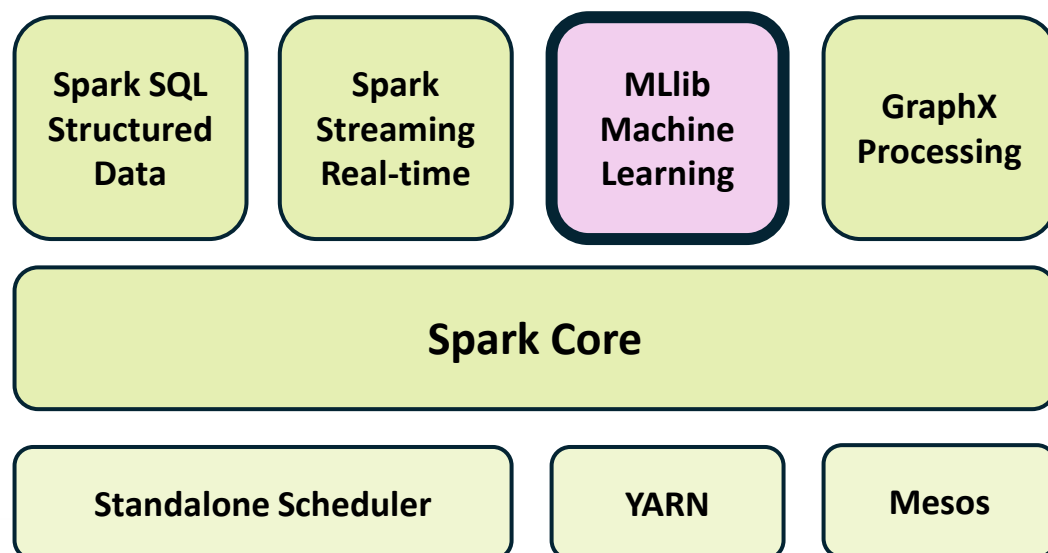


- Big Data Analysis
- Machine Learning
- Spark MLlib: talk about a specific platform for big data analysis
 - Architecture of Spark MLlib
 - What is in MLlib?
 - ML Algorithms
 - Why Spark MLlib?
 - Fundamentals of Spark MLlib
 - K-Means Clustering

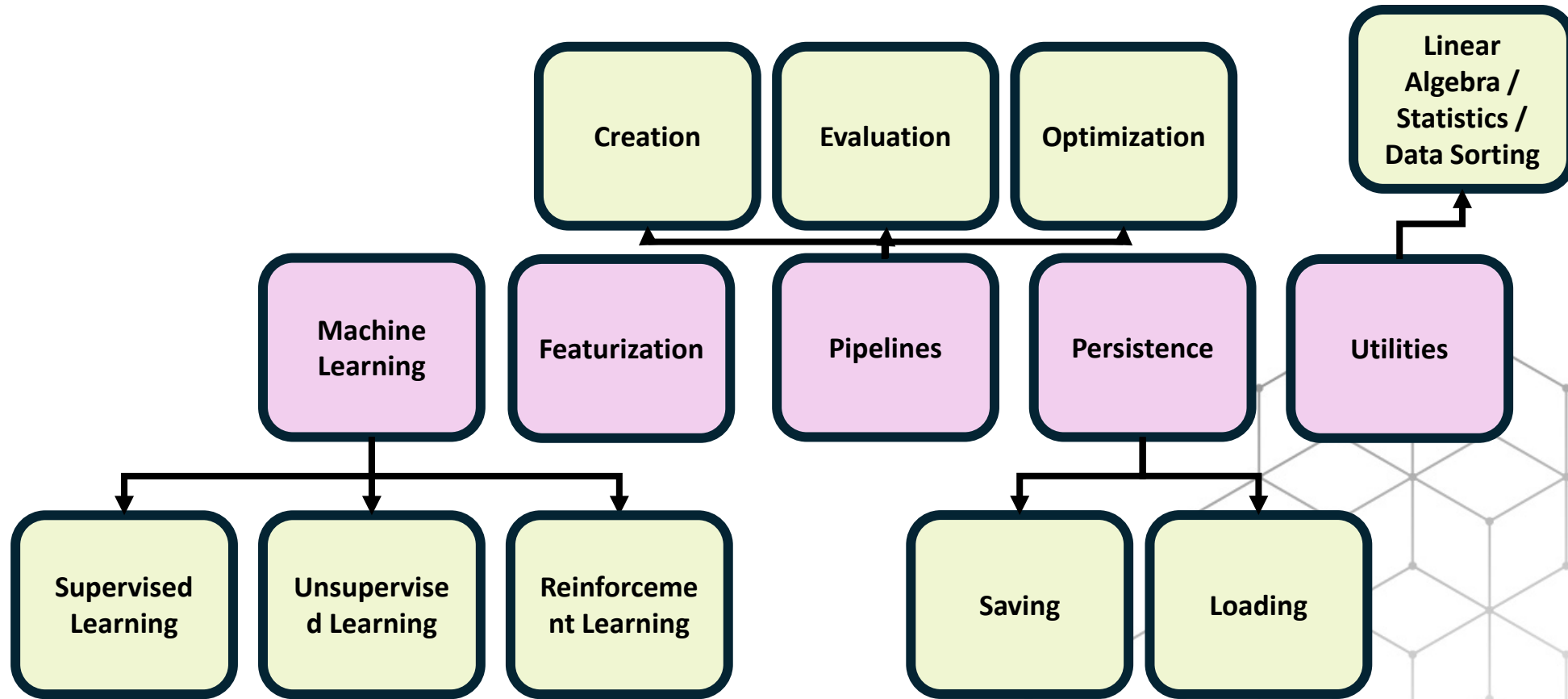


Architecture of Spark MLlib

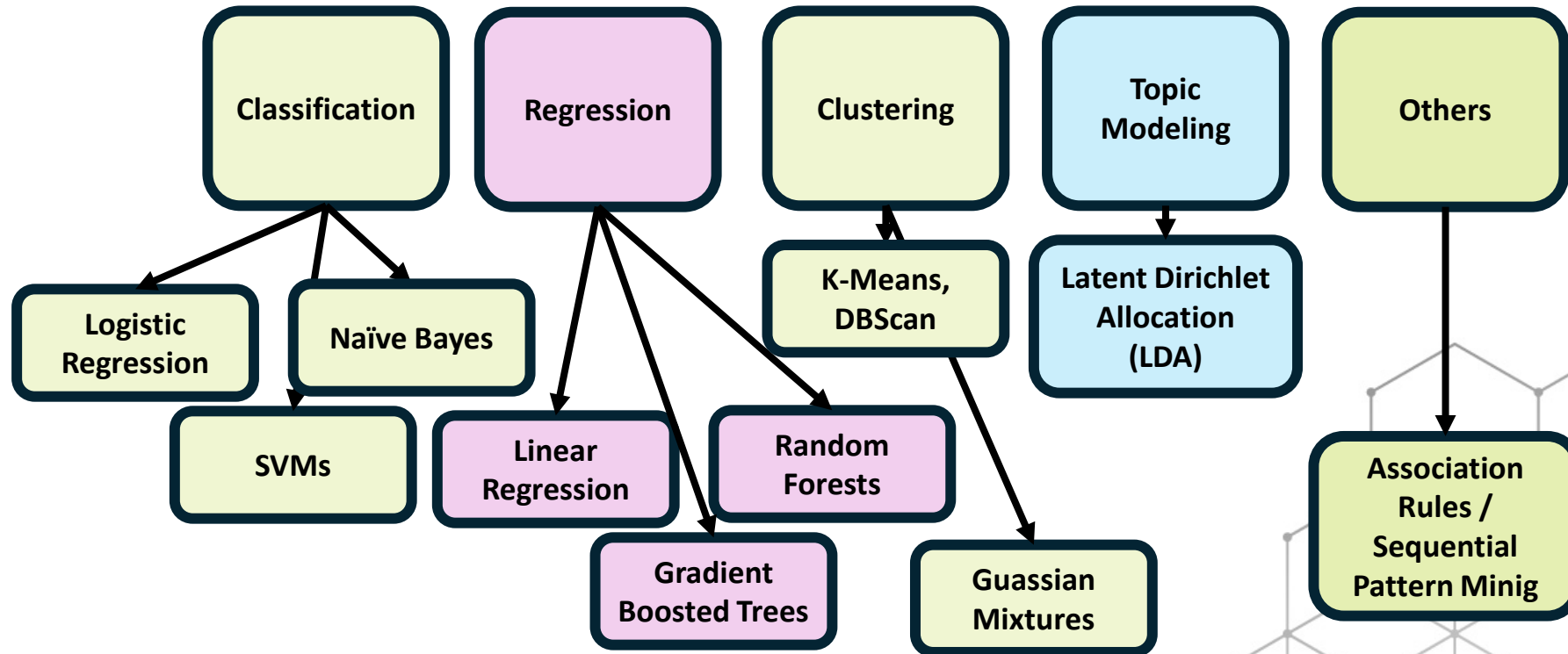
- Apache Spark's machine learning library.
- API in Java, Scala, Python, and R
- Make machine learning scalable and easy



What is in MLlib?

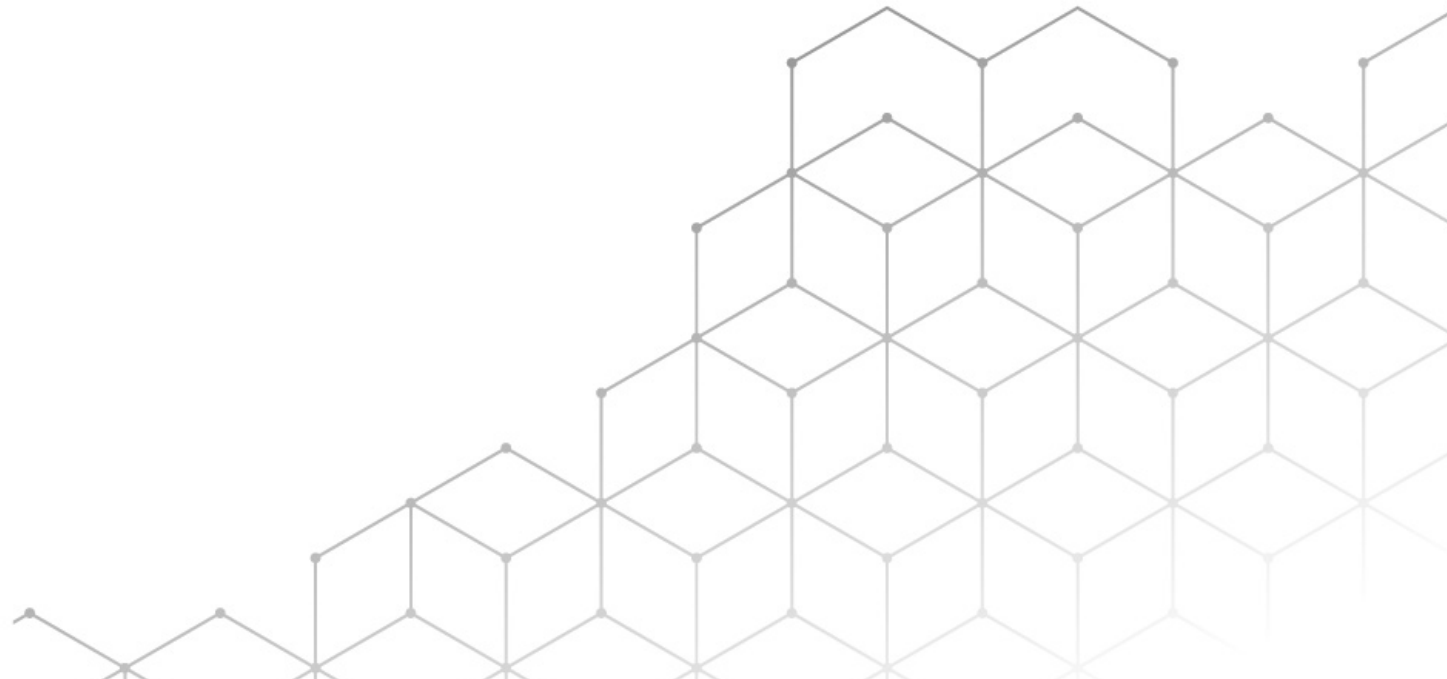


Machine Learning Algorithms



Why Spark MLlib?

- Simplicity
- Scalability
- Language Support



Example of Classification (1)

```
from pyspark.sql import SparkSession
from pyspark.ml.feature import VectorAssembler
from pyspark.ml.classification import DecisionTreeClassifier
from pyspark.ml.evaluation import MulticlassClassificationEvaluator
```

Step 1: Create Spark Session

```
spark = SparkSession.builder.appName("MLlib Example: Employee Attrition").getOrCreate()
```

Step 2: Load Data

```
data = spark.read.csv("employee_data.csv", header=True, inferSchema=True)
```

Step 3: Feature Engineering

```
assembler = VectorAssembler(
    inputCols=["Age", "YearsAtCompany", "JobSatisfaction"],
    outputCol="features"
)
data = assembler.transform(data).select("features", "Attrition")
```

Example of Classification (2)

Step 4: Split Data

```
train_data, test_data = data.randomSplit([0.8, 0.2], seed=42)
```

Step 5: Train Model

```
dt = DecisionTreeClassifier(featuresCol="features", labelCol="Attrition")  
model = dt.fit(train_data)
```

Step 6: Make Predictions

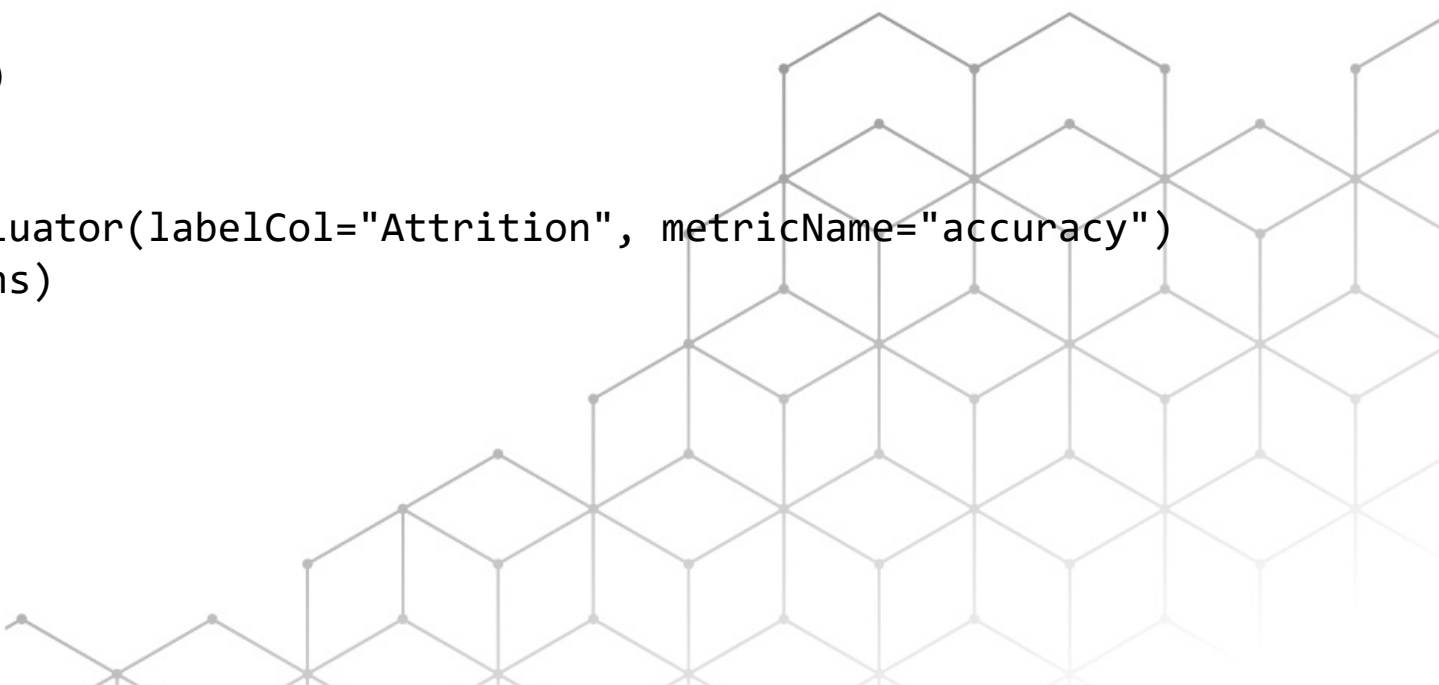
```
predictions = model.transform(test_data)
```

Step 7: Evaluate

```
evaluator = MulticlassClassificationEvaluator(labelCol="Attrition", metricName="accuracy")  
accuracy = evaluator.evaluate(predictions)  
print(f"Test Accuracy: {accuracy:.2f}")
```

Stop Spark Session

```
spark.stop()
```



Example of Clustering

```
from pyspark.sql import SparkSession
from pyspark.ml.clustering import KMeans
from pyspark.ml.evaluation import ClusteringEvaluator

# Step 1: Create Spark Session
spark = SparkSession.builder.appName("MLlib Example: K-Means Clustering").getOrCreate()

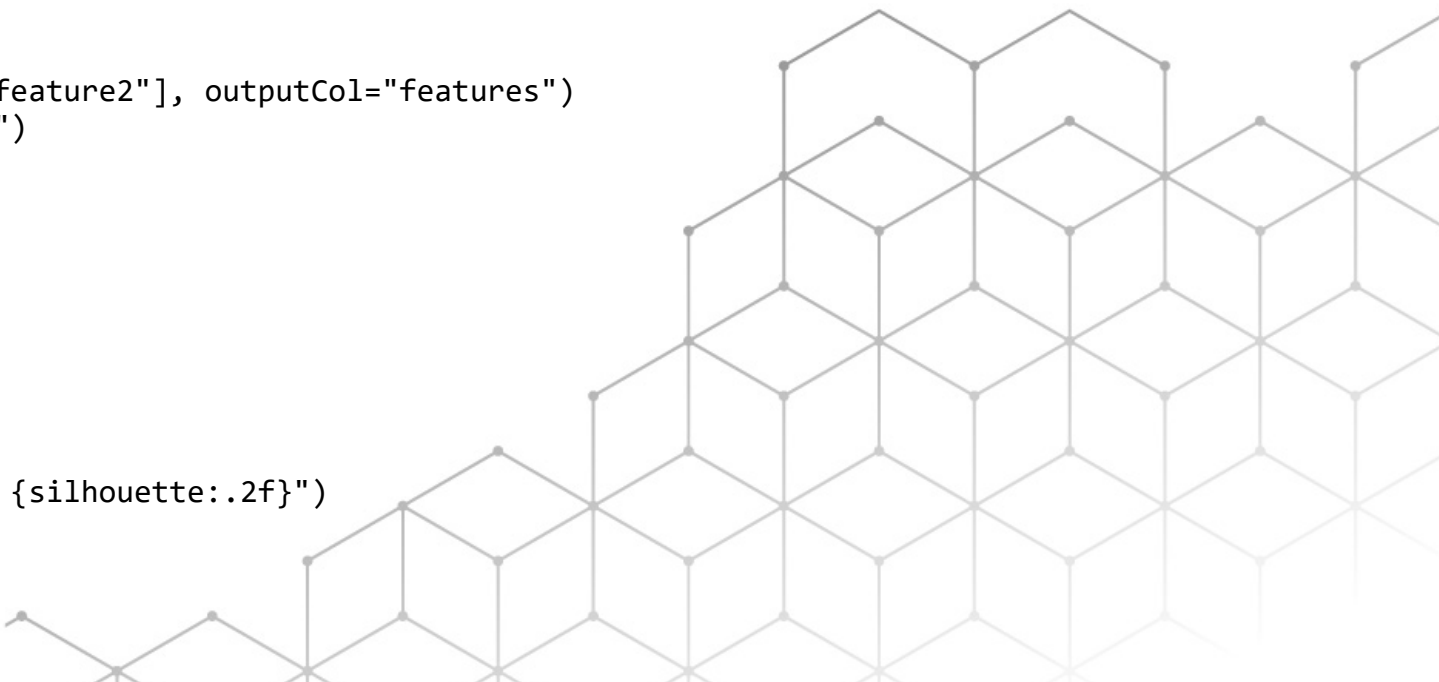
# Step 2: Load Data
data = spark.read.csv("data.csv", header=True, inferSchema=True)

# Step 3: Prepare Features
from pyspark.ml.feature import VectorAssembler
assembler = VectorAssembler(inputCols=["feature1", "feature2"], outputCol="features")
dataset = assembler.transform(data).select("features")

# Step 4: Train K-Means Model
kmeans = KMeans().setK(3).setSeed(1)
model = kmeans.fit(dataset)

# Step 5: Evaluate Clustering
predictions = model.transform(dataset)
evaluator = ClusteringEvaluator()
silhouette = evaluator.evaluate(predictions)
print(f"Silhouette with squared euclidean distance = {silhouette:.2f}")

# Stop Spark Session
spark.stop()
```



Spark SQL + MLlib

Step 1: Load Data with Spark SQL

```
query_result = spark.sql("SELECT feature1, feature2, label FROM table  
WHERE label IS NOT NULL")
```

Step 2: Prepare Features for MLlib

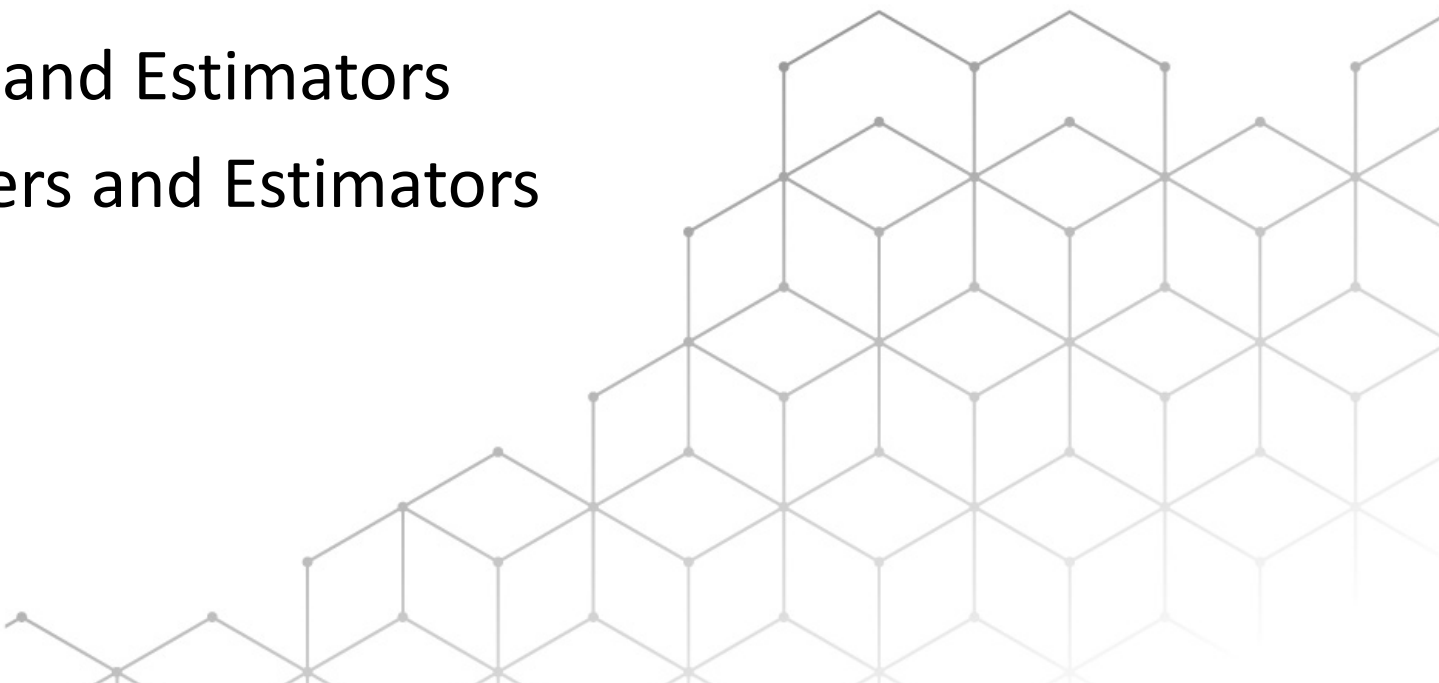
```
assembler = VectorAssembler(inputCols=["feature1", "feature2"],  
outputCol="features")  
dataset = assembler.transform(query_result).select("features", "label")
```

Step 3: Train a Logistic Regression Model

```
lr = LogisticRegression(featuresCol="features", labelCol="label")  
model = lr.fit(dataset)
```

Main concepts

- **DataFrame:** data type from Spark SQL allowing parallelization.
- **Transformer:** Algorithm that transforms one DataFrame into another
- **Estimator:** Machine Learning algorithm that can be fitted on a DataFrame, returning a model
- **Parameter:** for Transformers and Estimators
- **Pipeline:** Chain of Transformers and Estimators



DataFrames

- We have seen them before
- Conceptually Similar to Tables in relational databases
- Inspired by data frames in R and Pandas
- Supports a variety of data types: vectors, text , images and structured data
- Data can be accessed in columns
- Columns has names (“features” and “label”)

Person	Smoke	Disease
Ana	Yes	1
Maria	No	1
Joana	No	0
Raul	No	0

Transformers

- Algorithms, which Transform one DataFrame into another
 - generally by appending one or more columns
- Transformers
 - Feature transformers
 - Tokenization, normalization, hashing
 - Learned models
 - Results from Estimators

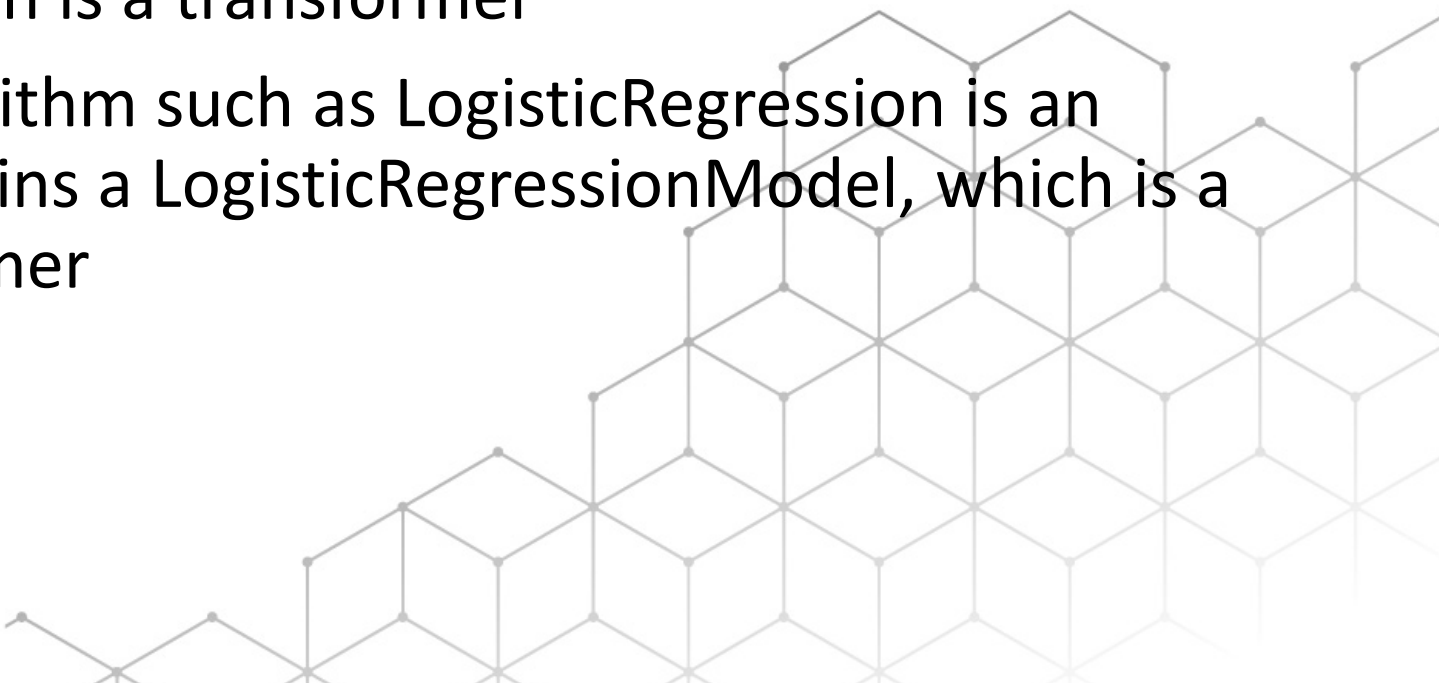
Person	Cigarettes	Disease
Ana Julia	4	1
Maria M.	2	1
Joana B.	2	0
Raul Dois	0	0



First Name	Last Name	Cigarettes	Disease
Ana	Julia	1	1
Maria	M.	0.5	1
Joana	B.	0.5	0
Raul	Dois	0	0

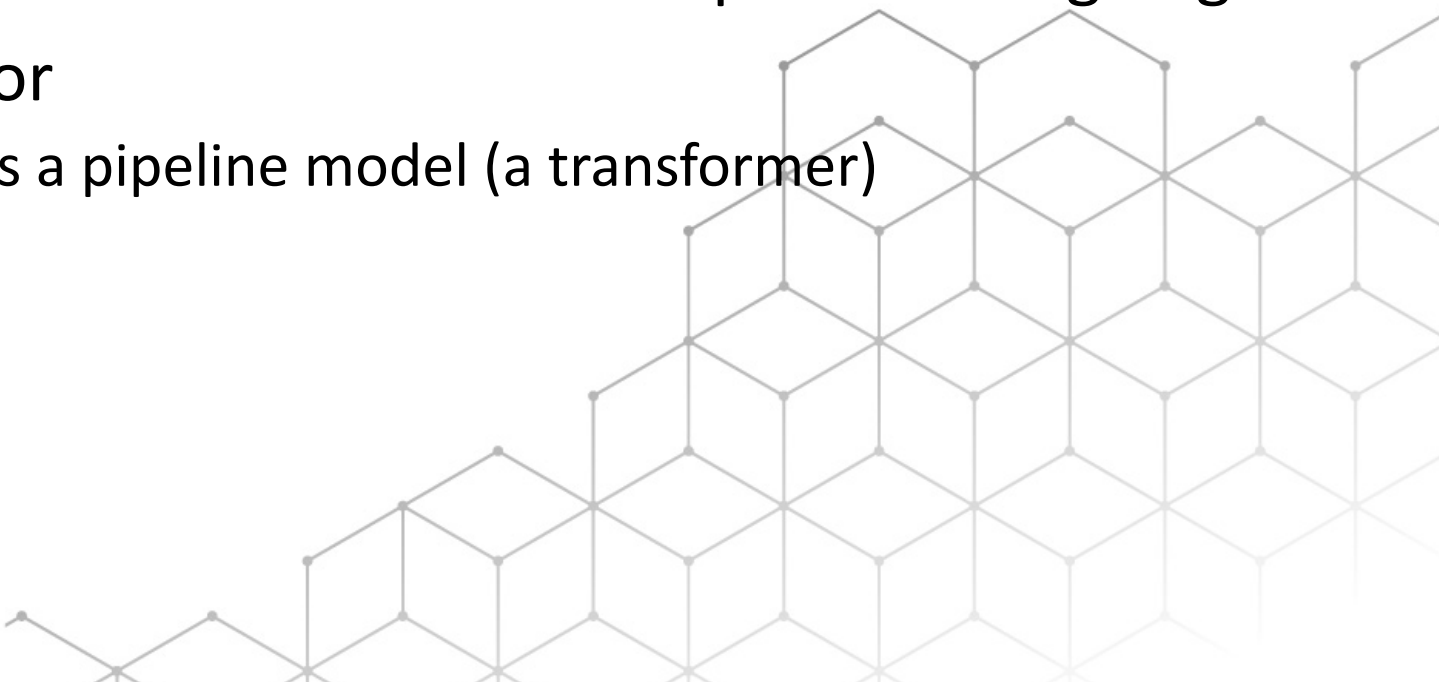
Estimator

- Learning Algorithm
- Gets fitted on training data
- Have a `fit()` method that receives a `DataFrame` as input
- Return a trained model, which is a transformer
- For example, a learning algorithm such as `LogisticRegression` is an Estimator, and calling `fit()` trains a `LogisticRegressionModel`, which is a Model and hence a Transformer

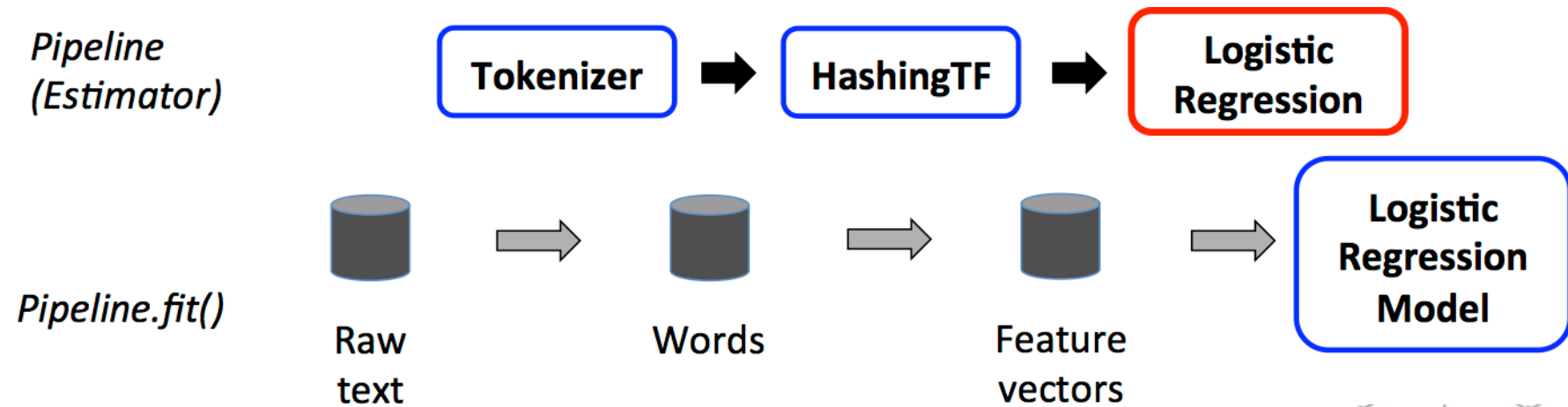


Pipeline

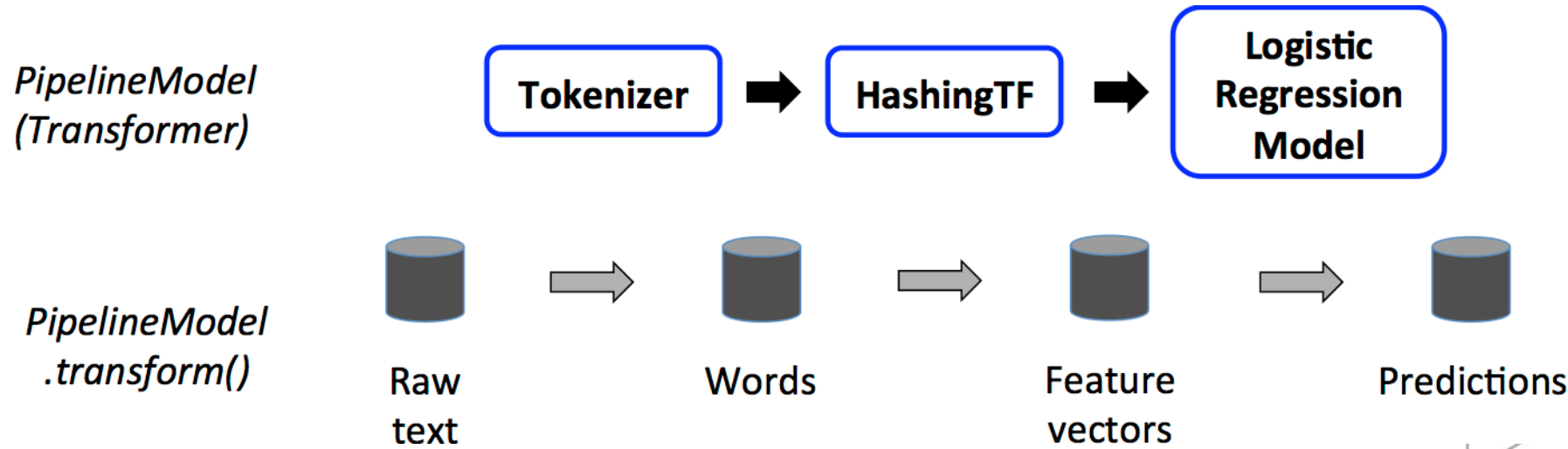
- Sequence of Transformers and Estimators
 - Each called an Stage
- Inspired by scikit-learn
- Estimators are fitted and return a transformer to keep the chain going
- A pipeline itself is an Estimator
 - Gets a DataFrames and returns a pipeline model (a transformer)



Example of Pipeline Training Stage



Example of Pipeline Testing Stage



K-Fold Cross Validation

Iteration 01



Iteration 02



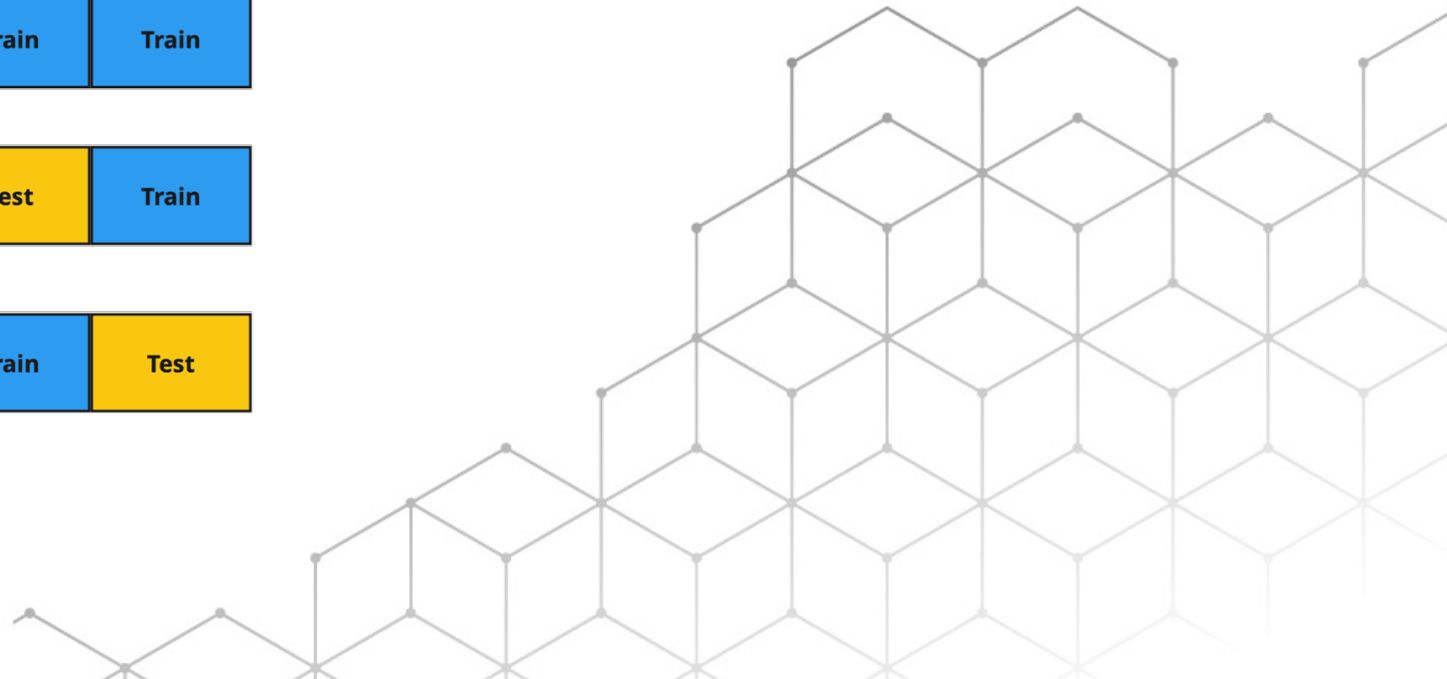
Iteration 03



Iteration 04



Iteration 05



Cross-Validation in Spark

- Cross-validation splits the data into multiple folds to train and validate the model with different hyperparameter combinations. It ensures a robust evaluation of model performance.
- Steps:
 - 1. Define a parameter grid for hyperparameter tuning.
 - 2. Use a `BinaryClassificationEvaluator`.
 - 3. Set up `CrossValidator` with pipeline, evaluator, and parameter grid.

Next Class

- NoSQL Databases



Questions and Thank you

